PCA MHEALTH · OPERATIONS MANUAL

How it was built, and how to run it yourself

The backend, the databases, Docker, the AI service, the chatbot, the phone app and the admin website: how each was made, and how to start, check, change and repair it without an AI assistant.

AI-Driven Mobile Health Platform for Prostate Cancer Diagnosis (Zambia)
Final-year research prototype · made on 14 October 2026 · code pictures from commit e106c34 with uncommitted changes
Not a medical device. All data shown is synthetic.

This manual explains every part of the system (the backend, the databases, Docker, the AI service, the chatbot, the phone app and the admin website): how each was made, and how to start, stop, check, change and repair it **without an AI assistant**.

- **Who it is for:** the project owner, and anyone who takes the project over.
- **What you need to know already:** how to open PowerShell and how to open a file in an editor. Everything else is written out.
- **Where commands are run:** in PowerShell, from the project folder `D:\Final Year Project\pca-mhealth`, unless a step says otherwise.
- **The pictures are made from the project itself** (section 17). The code pictures show the real files, each with its file name and line numbers, so you can open the same place yourself; the cover of the PDF says from which commit they were taken. The terminal pictures show the real output of the commands. The phone and admin pictures use synthetic data.
- **All data is synthetic.** The system is a research prototype, not a medical device.

Contents

1. The system on one page
2. What is installed on this PC
3. Starting and stopping
4. The settings file (.env)
5. The backend
6. The databases
7. Docker
8. The AI service
9. The chatbot, end to end
10. The phone app
11. The admin website
12. Tests and the quality gate
13. Git and GitHub
14. When something goes wrong
15. Before real patients
16. Where to read more
17. Updating this manual

---

1. The system on one page

PCa mHealth helps clinics in Zambia screen for prostate cancer. Clinicians register patients and record screenings on a phone, also without internet. Pathologists review slides. Patients see their own results, read or listen to health information, and ask an assistant questions. Administrators manage accounts on a website.

The pieces

Piece	What it does	Built with	Folder	Address on this PC
Phone app	Screens for patients, clinicians and pathologists; works offline	Flutter 3.35 (Dart)	`1-presentation-layer/mobile-app`	runs on the emulator or a phone
Admin website	Accounts, roles, patient links, audit log, dashboard	React 19 + Vite 8 (TypeScript)	`1-presentation-layer/admin-panel-web`	http://localhost:5173
Backend (the API)	Every rule of the system: sign-in, permissions, patients, consent, uploads, sync, chat, audit	NestJS 11 (TypeScript) on Node 24	`3-application-logic/backend`	http://localhost:3000
AI service	The AI models (labelled mock models for now) and the chatbot's search	FastAPI (Python 3.14)	`4-ai-intelligence-layer/ai-services`	http://127.0.0.1:8000
PostgreSQL	Accounts, patients, screenings, consents, audit log	PostgreSQL 18, installed on the PC	schema in `3-application-logic/backend/prisma`, history in `5-data-persistence/postgresql/migrations`	localhost:5432
MongoDB	AI reports, chats, scan details	MongoDB 8.3, installed on the PC	`3-application-logic/backend/src/persistence/mongo`	localhost:27017
Redis, MinIO, Qdrant	Shared rate limits; S3 file storage; vector search (planned)	Docker containers	`6-infrastructure/docker`	6379; 9000 and 9001; 6333

How a request travels

```
Phone app / admin website
        |   HTTPS in production; http://localhost:3000 on this PC
        v
Backend (NestJS)
  1. rate limit per network address
  2. who are you?  (JWT access token)
  3. rate limit per account
  4. may you do this?  (role permissions)
  5. is the request well-formed?  (validation)
  6. the service does the work and writes an audit entry
        |                    |                    |
        v                    v                    v
  PostgreSQL 18         MongoDB 8.3          AI service (FastAPI)
  (records)             (documents)          (models, chatbot search)
```

The phone app never talks to a database or to the AI service. Everything goes through the backend, so every rule is enforced in one place.

Why these choices (each has a decision record in `docs/decisions/`): - Flutter, because one codebase gives Android and iPhone apps with good offline support (ADR-001). - NestJS with PostgreSQL and MongoDB, and a separate Python service for AI, because AI libraries live in Python while the rest benefits from TypeScript's type checks (ADR-003). - A website for administrators instead of phone screens, because administration is desk work (ADR-005). - A chatbot that only answers from reviewed documents, because a health assistant must never invent medical text (ADR-009, ADR-010).

2. What is installed on this PC

Checked on 30 September 2026:

Tool	Version	Used for	How to check
Node.js / npm	24.15.0 / 11.12.1	backend, admin website, scripts	`node -v`
Python	3.14.5	AI service	`python --version`
Flutter / Dart SDK	3.35.2 / 3.9	phone app	`flutter --version`
Android Studio + SDK	emulator `Galaxy_S9_Plus_API_29`	running the app	Android Studio › Device Manager
PostgreSQL	18.6 (service `postgresql-x64-18`), with pgAdmin 4	main database	`& "C:\Program Files\PostgreSQL\18\bin\psql.exe" --version`

Tool	Version	Used for	How to check
MongoDB	8.3.2 (service `MongoDB`), with Compass	document database	Services (`services.msc`)
Docker Desktop	29.8.0	Redis, MinIO, Qdrant	`docker --version`
Git	2.51	version control	`git --version`
Google Chrome		admin website	

On a new PC, install the same tools, then follow section 3.5.

3. Starting and stopping

3.1 Start everything with one command

```
cd "D:\Final Year Project\pca-mhealth"
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-up.ps1
```

The script does this, in order:

1. Checks that the `.env` settings file exists.
2. Checks that the two database services (**postgresql-x64-18** and **MongoDB**) are running, and starts them if not.
3. Starts Docker Desktop if needed, then Redis, MinIO and Qdrant.
4. Stops an old backend if one is still running.
5. Applies any new database changes (`prisma migrate deploy`) and makes sure the synthetic demo data exists (`db:seed`).
6. Builds the backend and starts it in its own window.
7. Waits until the backend answers.
8. Starts the admin website in its own window.
9. Starts the AI service in its own window.

Keep the three windows open. Closing a window stops that piece.

This is the part of the script that looks after the databases:

6-infrastructure/scripts/dev-up.ps1 lines 30–43

```
# The databases installed on this PC. Both services start with Windows; this
# starts them if they were stopped (that needs an administrator PowerShell).
if (-not $DockerDatabases) {
  foreach ($name in @('postgresql-x64-18', 'MongoDB')) {
    $service = Get-Service -Name $name -ErrorAction SilentlyContinue
    if (-not $service) { Fail "The $name service is not installed. See docs\local-databases.md, or add -DockerDatabases." }
    if ($service.Status -ne 'Running') {
      Say "Starting the $name service..."
      try { Start-Service -Name $name } catch {
        Fail "Could not start $name. Open PowerShell as administrator and run: Start-Service $name"
      }
    }
  }
}
```

3.2 Check that everything is up

Open this	You should see
http://localhost:3000/api/v1/health	`"status":"ok"`
http://127.0.0.1:8000/v1/health	`"status":"ok"`; `chat_writer` says `quotes` (or `claude:…` when Claude is switched on); `chat_retrieval` says `keywords+meaning (MedCPT)` a few seconds after start-up (section 9.4)
http://localhost:5173	the admin sign-in page
http://localhost:3000/api/docs	the API explorer, listing every route

Is everything up?

```
PS D:\Final Year Project\pca-mhealth> curl.exe -s http://localhost:3000/api/v1/health
{"status":"ok","service":"pca-mhealth-backend","timestamp":"2026-10-01T11:56:12.009Z"}

PS D:\Final Year Project\pca-mhealth> curl.exe -s http://127.0.0.1:8000/v1/health
{"status":"ok","service":"pca-mhealth-ai-services","mode":"development-
mock","clinical_models_loaded":false,"chat_writer":"quotes","chat_retrieval":"keywords+meaning (MedCPT)"}
```

3.3 Stop everything

```
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-down.ps1
```

It stops the backend, the admin website, the AI service and the Docker containers. **No data is lost.** PostgreSQL and MongoDB keep running as Windows services, which is normal.

3.4 Start each piece by hand

Use this when you want to see one piece's messages, or when the script fails.

Databases (usually already running):

```
Get-Service postgresql-x64-18, MongoDB          # both should say Running
Start-Service postgresql-x64-18                 # needs PowerShell "Run as administrator"
Start-Service MongoDB
```

Docker services:

```
docker compose -f 6-infrastructure\docker\docker-compose.yml --env-file .env up -d
```

Backend:

```
cd 3-application-logic\backend
npm ci                      # only the first time, or after package.json changed
npx prisma migrate deploy   # apply database changes
npm run db:seed             # synthetic demo data (safe to repeat)
npm run build
npm run start:prod          # or: npm run start:dev  (restarts when you save a file)
```

AI service:

```
cd 4-ai-intelligence-layer\ai-services
py -3 -m venv .venv                                 # only the first time
.venv\Scripts\python -m pip install -e ".[dev]"     # only the first time
.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Admin website:

```
cd 1-presentation-layer\admin-panel-web
npm ci          # only the first time
npm run dev     # then open http://localhost:5173
```

Phone app: see section 10.2.

3.5 Setting up on a new PC

1. Install the tools in section 2.
2. Get the code: `git clone https://github.com/deedeestephen/final-year-project.git`
3. Create the settings file with fresh random secrets: `node 6-infrastructure\scripts\gen-keys.mjs --init-env`
4. Create the database login and database: follow steps 1 and 3 of local-databases.md, and put the same passwords in `.env`.
5. Install the AI service (the two "only the first time" lines above).
6. Run `dev-up.ps1`. It installs the other packages, creates the tables and adds the demo data.

3.6 Signing in

The seed creates four synthetic demo accounts:

Account	Role	Where it signs in
`admin@demo.pca-mhealth.test`	Administrator	admin website
`clinician@demo.pca-mhealth.test`	Clinician	phone app
`pathologist@demo.pca-mhealth.test`	Pathologist	phone app
`patient@demo.pca-mhealth.test`	Patient	phone app

- **The password** of all four is the value of `SEED_DEMO_PASSWORD` in `.env`.
- The first time an account signs in, it may be asked to choose its own password (12 or more characters).
- **Forgot it, or locked out?** Five wrong tries lock an account for 15 minutes. This command puts all four back to the password in `.env` and removes the lock:

```
powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\dev-up.ps1 -ResetDemoPasswords
```

4. The settings file (.env)

`.env` sits in the project folder. It holds every setting and every secret: database passwords, the keys that sign sessions, the key that encrypts personal fields.

- **It is never committed to Git.** `.gitignore` excludes it, and a secret scanner checks every commit.
- `.env.example` **is not the settings file.** It is a template with empty or placeholder values, which *is* in Git, so a new developer knows which settings exist.
- **How it was made:** `node 6-infrastructure\scripts\gen-keys.mjs --init-env` copies the template and fills every secret with random values.
- **After changing** `.env`**, restart the piece that reads it** (the backend or the AI service).
- **A value that contains** `$` **must be in single quotes**, for example `SEED_DEMO_PASSWORD='$example'`. Otherwise Docker reads `$example` as a variable and prints a warning.

.env.example lines 13–33

```
# --- PostgreSQL ---
# The project uses the PostgreSQL 18 installed on the PC (Windows service, port 5432);
# docs/local-databases.md shows how the pca login and database are created.
POSTGRES_USER=pca
POSTGRES_PASSWORD=change-me
POSTGRES_DB=pca_mhealth
DATABASE_URL=postgresql://pca:change-me@localhost:5432/pca_mhealth?schema=public
# The admin (postgres) password chosen when PostgreSQL was installed. Only the set-up
# steps in docs/local-databases.md use it; the app never does. Leave empty if not needed.
LOCAL_POSTGRES_ADMIN_PASSWORD=
# Only for the optional Docker database (dev-up.ps1 -DockerDatabases): its host port.
# Then DATABASE_URL must use this port too.
POSTGRES_PORT=5433

# --- MongoDB ---
# The project uses the MongoDB installed on the PC (Windows service, port 27017).
MONGO_USER=pca
MONGO_PASSWORD=change-me
MONGO_URL=mongodb://pca:change-me@localhost:27017/pca_mhealth?authSource=admin
# Only for the optional Docker database (dev-up.ps1 -DockerDatabases); then MONGO_URL uses it.
MONGO_PORT=27018
```

The settings in groups

Group	Settings	Meaning
Backend	`PORT`, `CORS_ORIGINS`, `RATE_LIMIT_MAX`, `USER_RATE_LIMIT_MAX`	port 3000; which website may call the API; 600 requests a minute per address, 120 per account
PostgreSQL	`DATABASE_URL`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `LOCAL_POSTGRES_ADMIN_PASSWORD`	where the main database is; the app's login; the admin login, used only for set-up
MongoDB	`MONGO_URL`, `MONGO_USER`, `MONGO_PASSWORD`	where the document database is
Redis	`REDIS_URL`, `REDIS_PASSWORD`	shared rate-limit counters; without it each backend counts alone
Files	`STORAGE_DRIVER`, `LOCAL_STORAGE_ROOT`, `S3_*`, `MINIO_*`	`local` (now): files go to `backend/var/objects`; `s3`: files go to MinIO
Sign-in	`JWT_PRIVATE_KEY_BASE64`, `JWT_PUBLIC_KEY_BASE64`	the key pair that signs sessions
Encryption	`FIELD_ENCRYPTION_KEY_BASE64`, `FIELD_HMAC_KEY_BASE64`	encrypt names, phone and NRC numbers. **If these are lost, those fields cannot be read again: back up `.env` safely**
AI	`AI_SERVICE_URL`, `AI_SERVICE_TOKEN`, `ANTHROPIC_API_KEY`, `CHAT_LLM_MODEL`, `CHAT_LLM_DAILY_LIMIT`	where the AI service is; the shared token; Claude is off while the key is empty
Push	`FCM_SERVICE_ACCOUNT_FILE`, `PUSH_POLL_MS`, `PUSH_MAX_AGE_MIN`	the Firebase key file (empty: push is off, section 18); how often new notifications are looked for; how old one may be and still be pushed

Group	Settings	Meaning
Chatbot search	`CHAT_RETRIEVAL`, `CHAT_MODELS_DIR`, `CHAT_MEANING_SLOTS`, `CHAT_MEANING_WAIT_MS`	`auto`: the meaning search is used when its models are downloaded (`keywords` switches it off); where the models are; how many questions it handles at once, and how long one waits before keywords alone answer (section 9.4)
Demo	`SEED_DEMO_PASSWORD`	the password of the four demo accounts

5. The backend

5.1 What it is and how it is organised

The backend is a NestJS application: a web server that answers requests under `http://localhost:3000/api/v1/…`. Its source is in `3-application-logic/backend/src`, in three folders that match the architecture:

Folder	What is inside
`gateway/`	What every request passes through: sign-in check, permissions, rate limits, validation, error format, logging, upload checks
`services/`	One folder per feature: `auth`, `users`, `patients`, `clinical`, `imaging`, `ai`, `chatbot`, `sync`, `notifications`, `fhir`, `admin`, `audit`
`persistence/`	Talking to stores: `database` (PostgreSQL through Prisma, and MongoDB), `storage` (files), `crypto` (field encryption), `mongo` (collection rules), `seed.ts` (demo data)

Each feature folder has the same three kinds of file: - a **controller** (`*.controller.ts`): the routes, and the permission each one needs; - a **service** (`*.service.ts`): the rules and the database work; - a **DTO** file (`*.dto.ts`): the exact shape of what may be sent and what comes back.

5.2 How it starts

`main.ts` is the first file that runs. It reads `.env`, builds the application, applies the security settings and starts listening on the port.

3-application-logic/backend/src/main.ts lines 1–24

```ts
import { NestFactory } from '@nestjs/core';
import type { NestExpressApplication } from '@nestjs/platform-express';
import { Logger } from 'nestjs-pino';
import { AppModule } from './app.module';
import { APP_CONFIG, type AppConfig } from './config/app-config';
import { configureApp } from './gateway/configure-app';
import { setupOpenApi } from './gateway/openapi';
import { loadRootEnv } from './config/repo-root';

async function bootstrap() {
  // Local development reads the repository-root .env; deployments inject variables.
  loadRootEnv();

  const app = await NestFactory.create<NestExpressApplication>(AppModule, {
    bufferLogs: true,
    bodyParser: false, // re-enabled with explicit size limits in configureApp
  });
  app.useLogger(app.get(Logger));
  const config = app.get<AppConfig>(APP_CONFIG);
  configureApp(app, config);
  if (config.apiDocsEnabled) setupOpenApi(app);
  await app.listen(config.port);
}
void bootstrap();
```

5.3 Four guards on every request

`app.module.ts` registers four guards that run on **every** route, in this order. A route is closed unless the code says otherwise ("deny by default").

3-application-logic/backend/src/app.module.ts lines 86–98

```ts
  providers: [
    // Gateway concerns applied to every route, in this order, in tests exactly as in production:
    // address rate limit -> authenticate (deny by default) -> per-account
    // rate limit -> authorise (RBAC).
    { provide: APP_GUARD, useClass: AppThrottlerGuard },
    { provide: APP_GUARD, useClass: JwtAuthGuard },
    { provide: APP_GUARD, useClass: UserRateLimitGuard },
    { provide: APP_GUARD, useClass: PermissionsGuard },
    { provide: APP_FILTER, useClass: AllExceptionsFilter },
    { provide: APP_PIPE, useFactory: createValidationPipe },
  ],
})
export class AppModule {}
```

1. **AppThrottlerGuard:** at most 600 requests a minute from one network address.
2. **JwtAuthGuard:** the request must carry a valid session token. Only routes marked `@Public()` (sign-in, health) skip this.
3. **UserRateLimitGuard:** at most 120 requests a minute per account.
4. **PermissionsGuard:** the account's roles must include the permission the route asks for.

The permissions guard compares what the route needs with what the user has, and writes an audit entry when it refuses:

3-application-logic/backend/src/gateway/access/permissions.guard.ts lines 29–44

```ts
async canActivate(context: ExecutionContext): Promise<boolean> {
  const targets = [context.getHandler(), context.getClass()];
  if (this.reflector.getAllAndOverride<boolean>(IS_PUBLIC, targets))
    return true;

  const required =
    this.reflector.getAllAndMerge<PermissionCode[]>(
      REQUIRED_PERMISSIONS,
      targets,
    ) ?? [];
  if (required.length === 0) return true;

  const req = context.switchToHttp().getRequest<AuthenticatedRequest>();
  const user = req.user;
  const missing = required.filter((p) => !user?.permissions.includes(p));
  if (missing.length === 0) return true;
```

- **Roles:** `PATIENT`, `CLINICIAN`, `PATHOLOGIST`, `ADMIN`.
- **Permissions** are codes such as `patient:read` or `user:manage`. Roles hold permissions, and users hold roles.
- **Every route and who may use it** is listed in access-matrix.md (73 routes). That file is generated from the code with `npm run access:matrix`, and the quality gate fails if it is out of date.

5.4 One feature from top to bottom: deleting an account

The **route** says which address it answers, which permission it needs, and what it returns:

3-application-logic/backend/src/services/users/users.controller.ts lines 103–119

```ts
  /** Only accounts without clinical history; the others can be disabled. */
  @Delete(':id')
  @HttpCode(HttpStatus.NO_CONTENT)
  @RequirePermissions('user:manage')
  @ApiNoContentResponse({ description: 'The account was deleted' })
  @ApiConflictResponse({
    description:
      'HAS_CLINICAL_HISTORY: the account has clinical history; disable it instead',
  })
  remove(
    @Param('id', new ParseUUIDPipe()) id: string,
    @CurrentUser() actor: AuthenticatedUser,
    @Ctx() ctx: RequestContext,
  ): Promise<void> {
    return this.users.delete(id, actor, ctx);
  }
}
```

The **service** holds the rules. It refuses the administrator's own account and any account named in the clinical record. Otherwise it deletes the account and writes the audit entry in one database transaction (both happen or neither does). Last, it removes the account's chats from MongoDB.

3-application-logic/backend/src/services/users/users.service.ts lines 377–432

```ts
  async delete(
    id: string,
    actor: AuthenticatedUser,
    ctx: RequestContext,
  ): Promise<void> {
    if (id === actor.id) {
      throw new BadRequestException({
        code: 'SELF_DELETE',
        message: 'You cannot delete your own account',
      });
    }
    const existing = await this.prisma.user.findUnique({
      where: { id },
      include: { ...WITH_ROLES, patientProfile: { select: { id: true } } },
    });
    if (!existing)
      throw new NotFoundException({
        code: 'NOT_FOUND',
        message: 'User not found',
      });
    const history = await this.clinicalHistory(id);
    if (Object.values(history).some((n) => n > 0)) {
      throw new ConflictException({
        code: 'HAS_CLINICAL_HISTORY',
        message:
          'This account has clinical history, so it cannot be deleted. Disable it instead: it can no longer sign in, and the history stays traceable.',
        details: history,
      });
    }
    await this.prisma.$transaction(async (tx) => {
      await tx.user.delete({ where: { id } });
      await this.audit.record(
        {
          action: 'user.deleted',
          entityType: 'user',
          entityId: id,
          outcome: 'SUCCESS',
          actorUserId: actor.id,
          actorRole: actor.roles.join(','),
          details: {
            roles: existing.roles.map((r) => r.role.name).sort(),
            wasLinkedToPatient: existing.patientProfile !== null,
            synthetic: existing.isSynthetic,
          },
          ...ctx,
        },
        tx,
      );
    });
    await (
      await this.mongo.db()
    )
      .collection(CHAT_COLLECTION)
      .deleteMany({ userId: id });
  }
}
```

Every feature follows this pattern: controller → service → Prisma (database) → audit.

5.5 Security that is built in

Protection	How
Passwords	Stored only as Argon2id hashes (64 MB memory, 3 passes). Nobody, including an administrator, can read a password. At least 12 characters
Sessions	A short access token (15 minutes), signed with an Ed25519 key pair, plus a refresh token (14 days) that is replaced on every use. A stolen, reused refresh token ends the whole session family
Lockout	5 wrong passwords lock the account for 15 minutes
Personal fields	Names, phone and NRC numbers are encrypted in the database (AES-256-GCM). The NRC can still be matched exactly through a keyed hash
Audit log	Every important action is recorded. The table only accepts new rows, and each row carries a hash of the row before it, so any later change is detected
Input	Every request body is validated against its DTO; unknown fields are rejected
Uploads	Size limits, file-type checks by content, DICOM header checks

The audit chain is enforced by the database itself, not only by the application:

5-data-persistence/postgresql/migrations/20260923203337_constraints_and_audit/migration.sql lines 75–92

```
75 CREATE OR REPLACE FUNCTION audit_logs_before_insert()
76 RETURNS trigger LANGUAGE plpgsql AS $$
77 DECLARE
78   v_prev text;
79 BEGIN
80   PERFORM pg_advisory_xact_lock(hashtext('audit_logs_chain'));
81   NEW.seq := nextval(pg_get_serial_sequence('audit_logs', 'seq'));
82   NEW.occurred_at := date_trunc('milliseconds', coalesce(NEW.occurred_at, now()));
83   SELECT row_hash INTO v_prev FROM audit_logs ORDER BY seq DESC LIMIT 1;
84   NEW.prev_hash := coalesce(v_prev, repeat('0', 64));
85   NEW.row_hash := audit_logs_row_hash(NEW.prev_hash, NEW);
86   RETURN NEW;
87 END;
88 $$;
89
90 CREATE TRIGGER audit_logs_chain
91   BEFORE INSERT ON audit_logs
92   FOR EACH ROW EXECUTE FUNCTION audit_logs_before_insert();
```

On the admin website, **Audit log › Check integrity** re-computes the whole chain.

5.6 Backend commands

Run these in `3-application-logic\backend`:

Task	Command
Install packages	`npm ci`
Build	`npm run build`

Task	Command
Start (built code)	`npm run start:prod`
Start and restart on every save	`npm run start:dev`
Unit tests	`npm test`
End-to-end tests (no database needed)	`npm run test:e2e`
Database tests (need PostgreSQL, MongoDB and Docker's MinIO)	`npm run test:db`
Whole workflows on the live system	`npm run test:workflows`
Format and lint	`npx prettier --write src test` and `npm run lint`
Rebuild the API description	`npm run openapi:export`
Rebuild the route list	`npm run access:matrix`
Reset the demo passwords	`npm run db:reset-demo`

5.7 The API explorer

http://localhost:3000/api/docs lists every route with its inputs and outputs. To try one: sign in through `POST /api/v1/auth/login`, copy the `accessToken`, click **Authorize** and paste it. The same description is saved in `2-api-gateway/openapi/openapi.json`.

5.8 Adding a new route

1. Pick the feature folder in `src/services/` (or create one with a module, controller and service).
2. Describe the input and output in the `*.dto.ts` file.
3. Add the method to the service. Record an audit entry for anything that changes data.
4. Add the route to the controller with `@RequirePermissions('…')`.
5. Write a test: a unit test next to the file (`*.spec.ts`), or a database test in `test/db/`.
6. Run `npm run openapi:export` and `npm run access:matrix`.
7. Run the gate: `powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend`.

6. The databases

6.1 What is stored where

Data	Where	Why there
Accounts, roles, patients, screenings, consents, scans and slides (the records), AI jobs, notifications and the phones they are pushed to, sync history, audit log	PostgreSQL 18, database `pca_mhealth`, 21 tables	Structured records with strict rules and links between them
AI reports, AI logs, scan technical details, chatbot conversations	MongoDB 8.3, database `pca_mhealth`, 4 collections	Documents whose shape varies
Uploaded scan and slide files	the folder `3-application-logic/backend/var/objects`	Large files do not belong in a database
Rate-limit counters	Redis (Docker)	Fast, shared, temporary

psql: the tables in PostgreSQL 18

```
PS D:\Final Year Project\pca-mhealth> & "C:\Program Files\PostgreSQL\18\bin\psql.exe" -h localhost -U pca -d
pca_mhealth -c "\dt"
                List of tables
 Schema |           Name           | Type  | Owner
--------+--------------------------+-------+-------
 public | _prisma_migrations       | table | pca
 public | ai_jobs                  | table | pca
 public | ai_models                | table | pca
 public | audit_logs               | table | pca
 public | clinical_records         | table | pca
 public | consents                 | table | pca
 public | explainability_artifacts | table | pca
 public | facilities               | table | pca
 public | histopathology_specimens | table | pca
 public | imaging_studies          | table | pca
 public | notifications            | table | pca
 public | password_reset_tokens    | table | pca
 public | patients                 | table | pca
 public | permissions              | table | pca
 public | push_devices             | table | pca
 public | refresh_tokens           | table | pca
 public | role_permissions         | table | pca
 public | roles                    | table | pca
 public | sync_operations          | table | pca
 public | user_roles               | table | pca
 public | users                    | table | pca
(21 rows)
```

6.2 PostgreSQL: how the tables were made

The tables are **not** created by hand. They are described in one file, `3-application-logic/backend/prisma/schema.prisma`. This is the part that describes a user account:

3-application-logic/backend/prisma/schema.prisma lines 24–52

```
model User {
  id                String     @id @default(uuid()) @db.Uuid
  email             String     @unique // stored lower-cased by the application
  passwordHash      String     @map("password_hash")
  displayName       String     @map("display_name")
  status            UserStatus @default(ACTIVE)
  failedLoginCount  Int        @default(0) @map("failed_login_count")
  lockedUntil       DateTime?  @map("locked_until") @db.Timestamptz(3)
  lastLoginAt       DateTime?  @map("last_login_at") @db.Timestamptz(3)
  mustChangePassword Boolean   @default(false) @map("must_change_password")
  facilityId        String?    @map("facility_id") @db.Uuid
  isSynthetic       Boolean    @default(false) @map("is_synthetic")
  /// Patient self-registration: contact phone and identity document, encrypted
  /// (AES-256-GCM). The HMAC allows exact matching to a clinic record without
  /// storing the number in clear.
  phoneEnc          Bytes?     @map("phone_enc")
  idDocumentType    String?    @map("id_document_type") // NRC | PASSPORT
  idNumberEnc       Bytes?     @map("id_number_enc")
  idNumberHmac      String?    @unique @map("id_number_hmac")
  createdAt         DateTime   @default(now()) @map("created_at") @db.Timestamptz(3)
  updatedAt         DateTime   @updatedAt @map("updated_at") @db.Timestamptz(3)

  facility          Facility?          @relation(fields: [facilityId], references: [id])
  roles             UserRole[]
  refreshTokens     RefreshToken[]
  passwordResetTokens PasswordResetToken[]
  notifications     Notification[]
  pushDevices       PushDevice[]
  patientProfile    Patient?           @relation("PatientAccount")
```

From that file, the tool **Prisma** generates: - **migrations:** SQL files in `5-data-persistence/postgresql/migrations/`, one folder per change, in date order. They are the history of the database, and they are in Git. - **the Prisma client:** the typed code the backend uses to read and write (`this.prisma.user.findUnique(…)`).

To change the database:

1. Edit `schema.prisma`.
2. Create the migration and apply it to your database: `powershell cd 3-application-logic\backend npm run db:migrate:dev -- --name describe_the_change`
3. Read the new SQL file in the migrations folder. Special rules (such as the audit triggers) are written into it by hand.
4. Commit the schema and the migration together.

To apply existing migrations (done by `dev-up.ps1`): `npx prisma migrate deploy`. It only ever adds what is missing.

To check:

Prisma: is the database schema up to date?

```
PS D:\Final Year Project\pca-mhealth\3-application-logic\backend> npx prisma migrate status
Loaded Prisma config from prisma.config.ts.

Prisma config detected, skipping environment variable loading.
Prisma schema loaded from prisma\schema.prisma
Datasource "db": PostgreSQL database "pca_mhealth", schema "public" at "localhost:5432"

7 migrations found in prisma/migrations

Database schema is up to date!
```

> **Never run** `prisma migrate reset`**.** It deletes every table and all data.

6.3 MongoDB: collections with rules

MongoDB would accept anything, so each collection has a validator: a rule the server checks on every write. They are defined in `src/persistence/mongo/collections.ts` and applied with:

```
npm run db:mongo:migrate
```

Collection	Holds
`ai_reports`	the result of each AI analysis, with its "MOCK" or "RESEARCH_MODEL" label
`ai_inference_logs`	a technical log of each model run
`imaging_metadata`	technical fields of uploaded scans (never the patient's name)
`chatbot_conversations`	chats; deleted automatically 180 days after the last message

6.4 Looking at the data

- **PostgreSQL:** open **pgAdmin 4** › Servers › PostgreSQL 18 › Databases › `pca_mhealth` › Schemas › public › Tables. Right-click a table › View/Edit Data › All Rows.
- **MongoDB:** open **MongoDB Compass**, connect to `mongodb://localhost:27017`, open `pca_mhealth`.

Reading is always safe. **Do not add, change or delete rows by hand:** - A typed-in account cannot sign in, because passwords are stored as hashes and some fields are encrypted. - A hand-deleted row breaks links between records. - A hand-edited audit log fails its integrity check.

Use the admin website or the API instead. The full guide is local-databases.md.

6.5 Backups

```
# PostgreSQL: one file
& "C:\Program Files\PostgreSQL\18\bin\pg_dump.exe" -h localhost -U pca -d pca_mhealth -Fc -f
D:\backups\pca_mhealth.dump

# Restore into an empty database
& "C:\Program Files\PostgreSQL\18\bin\pg_restore.exe" -h localhost -U pca -d pca_mhealth --no-owner -
-no-privileges D:\backups\pca_mhealth.dump
```

- **MongoDB:** copy it to another server or database name with the project's tool (below), or install the free *MongoDB Database Tools* for `mongodump` and `mongorestore`.
- **Files:** copy the folder `3-application-logic/backend/var/objects`.
- `.env`**:** keep a copy somewhere safe. Without its encryption keys, the encrypted fields in a backup cannot be read.

The copy tool walks through every collection and copies its rules, its indexes and its documents:

3-application-logic/backend/tools/copy-mongo.ts lines 30–67

```ts
  for (const info of collections) {
    if (info.name.startsWith('system.')) continue;
    const exists = await target
      .listCollections({ name: info.name })
      .hasNext();
    if (!exists) {
      const options: Document = ('options' in info && info.options) || {};
      await target.createCollection(info.name, {
        validator: options.validator as Document | undefined,
        validationLevel:
          options.validationLevel as CreateCollectionOptions['validationLevel'],
        validationAction:
          options.validationAction as CreateCollectionOptions['validationAction'],
      });
    }
    const into = target.collection(info.name);
    if ((await into.estimatedDocumentCount()) > 0) {
      console.log(`${info.name}: already has documents, skipped`);
      continue;
    }
    for (const index of await source.collection(info.name).indexes()) {
      if (index.name === '_id_') continue;
      const {
        key,
        v: _v,
        ns: _ns,
        ...rest
      } = index as Record<string, unknown> & { key: Record<string, 1 | -1> };
      await into.createIndex(key, rest);
    }
    const docs = await source.collection(info.name).find().toArray();
    if (docs.length > 0) {
      await into.insertMany(docs, { bypassDocumentValidation: true });
    }
    const copied = await into.countDocuments();
    console.log(`${info.name}: ${docs.length} read, ${copied} in target`);
  }
} finally {
```

6.6 How the databases were moved onto this PC

Until 29 September 2026 PostgreSQL and MongoDB ran inside Docker. They were moved to the servers installed on the PC in six steps: create the login and database, dump and restore PostgreSQL, create the MongoDB login, copy the collections, change two lines in `.env`, and switch the Docker copies off. Every table had the same number of rows afterwards, and the audit chain still verified. The exact commands, and how to switch back, are in local-databases.md.

7. Docker

Docker runs a program in a sealed box called a **container**, with its data in a **volume** that survives restarts. This project uses Docker only for helper services, so nothing has to be installed for them.

`6-infrastructure/docker/docker-compose.yml` lists the containers:

6-infrastructure/docker/docker-compose.yml lines 1–50

```yaml
# Development data stores for PCa mHealth. Not a production deployment.
# Usage (from the repository root): docker compose -f 6-infrastructure/docker/docker-compose.yml --env-file .env up -d
#
# PostgreSQL and MongoDB normally run as the services installed on the PC
# (PostgreSQL 18 on 5432, MongoDB on 27017; docs/local-databases.md), so here
# they are in the optional "docker-db" profile. To use Docker for them instead,
# add --profile docker-db and point DATABASE_URL / MONGO_URL at 5433 / 27018.
name: pca-mhealth

services:
  postgres:
    profiles: [docker-db]
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-pca}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD in .env}
      POSTGRES_DB: ${POSTGRES_DB:-pca_mhealth}
    ports: ["127.0.0.1:${POSTGRES_PORT:-5433}:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-pca}"]
      interval: 5s
      retries: 10
    restart: unless-stopped

  mongo:
    profiles: [docker-db]
    image: mongo:7
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER:-pca}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:?set MONGO_PASSWORD in .env}
    ports: ["127.0.0.1:${MONGO_PORT:-27018}:27017"]
    volumes: [mongodata:/data/db]
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 5s
      retries: 10
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--requirepass", "${REDIS_PASSWORD:?set REDIS_PASSWORD in .env}"]
    ports: ["127.0.0.1:6379:6379"]
    restart: unless-stopped

  minio:
    image: quay.io/minio/minio@sha256:14cea493d9a34af32f524e538b8346cf79f3321eff8e708c1e2960462bd8936e
    command: ["server", "/data", "--console-address", ":9001"]
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER:-pcaminio}
```

Service	What it is	Port	Used now?
`redis`	fast counters for rate limits	6379	yes
`minio`	S3-style file storage, with a web console on 9001	9000, 9001	by the database tests; by the app only when `STORAGE_DRIVER=s3`

Service	What it is	Port	Used now?
`qdrant`	vector search	6333	not yet (`VECTOR_STORE=memory`)
`postgres` , `mongo`	the old Docker databases, kept as a backup	5433, 27018	no: they are in the optional `docker-db` profile and stay off

Commands (Docker Desktop must be running; wait for "Engine running"):

```
$compose = "6-infrastructure\docker\docker-compose.yml"

docker compose -f $compose --env-file .env up -d        # start
docker compose -f $compose --env-file .env ps           # what is running
docker compose -f $compose --env-file .env logs redis   # one service's messages
docker compose -f $compose --env-file .env restart minio
docker compose -f $compose --env-file .env stop         # stop, keep the data
```

Docker: what runs in containers

```
PS D:\Final Year Project\pca-mhealth> docker compose -f 6-infrastructure\docker\docker-compose.yml --env-file
.env ps --format "table {{.Service}}\t{{.Status}}\t{{.Ports}}"
SERVICE   STATUS        PORTS
minio     Up 25 hours   127.0.0.1:9000-9001->9000-9001/tcp
qdrant    Up 25 hours   127.0.0.1:6333->6333/tcp
redis     Up 25 hours   127.0.0.1:6379->6379/tcp
```

- `stop` **keeps everything.** `down` removes the containers but keeps the volumes. `down -v` **deletes the volumes and their data: do not use it** unless you mean to.
- **To use the Docker databases again:** set the ports in `DATABASE_URL` and `MONGO_URL` to 5433 and 27018, then run `dev-up.ps1 -DockerDatabases` .
- **A password change** for Redis or MinIO needs the container recreated: `docker compose -f $compose --env-file .env up -d --force-recreate redis minio` .

8. The AI service

A separate Python program (FastAPI) in `4-ai-intelligence-layer/ai-services` . Only the backend may call it, with the shared `AI_SERVICE_TOKEN` .

Route	What it does
`GET /v1/health`	says it is up, that the models are **development mocks**, whether Claude writes chat answers, and how chat passages are found
`GET /v1/models`	the list of models and their versions
`POST /v1/infer`	runs an analysis (MRI, slide, risk score)
`POST /v1/chat/answer`	finds the reviewed passages for a question and returns the answer

The models are mocks. No trained model exists yet, so the service returns labelled development data ("*DEVELOPMENT MOCK DATA — NOT A CLINICAL RESULT*"), and the app shows that label. When real models are trained, only this service changes: see ai-model-integration-guide.md.

Commands (in `4-ai-intelligence-layer\ai-services`):

Task	Command
Install (first time)	`py -3 -m venv .venv` then `.venv\Scripts\python -m pip install -e ".[dev]"`
Run	`.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000`
Tests	`.venv\Scripts\python -m pytest -q`
Lint and types	`.venv\Scripts\python -m ruff check .` and `.venv\Scripts\python -m mypy`
Download the meaning-search models (once, about 880 MB)	`.venv\Scripts\python -m app.chat.embeddings download`
Measure the assistant's search on the question sets	`.venv\Scripts\python -m app.chat.evaluate`

9. The chatbot, end to end

9.1 The path of one question

1. The person types or speaks a question in the app. Speech is turned into text by the phone.
2. The app sends the text to the backend: `POST /api/v1/chat/conversations/{id}/messages`.
3. The backend checks the permission (`chatbot:use`) and the limit (30 questions an hour per account).
4. **Safety rules** look at the question. An emergency, self-harm, a medicine dose, or a patient asking what their own results mean gets a fixed, careful reply. Nothing is looked up.
5. **Small talk** ("hello", "how are you", "tell me a joke") gets a friendly fixed reply.
6. Otherwise the backend asks the AI service, which **searches the reviewed knowledge base**: it finds the passages that share the question's key words, chooses the article closest in meaning, and the best-matching section of it.
7. If Claude is switched on, Claude writes a short answer **from those passages only**. If not, or if anything fails, the passages are quoted word for word.
8. The backend checks the answer: it must have a source and must not contain a dose.
9. Question and answer are stored in MongoDB. The audit log records that a question was asked, never its words.
10. The app shows the answer with its sources, the content's review status and a disclaimer.

Steps 4 to 8 are this one function:

3-application-logic/backend/src/services/chatbot/chatbot.service.ts lines 270–327

```ts
  /** Safety check, then the quoted answer, then the output check. */
  private async answer(
    text: string,
    audience: ChatAudience,
    language: string,
    earlier: StoredMessage[],
  ): Promise<{ answer: StoredMessage; reason: string }> {
    const fixed = (safety: ChatSafety, fixedText: string, reason: string) => ({
      answer: {
        id: randomUUID(),
        role: 'assistant' as const,
        text: fixedText,
        at: new Date(),
        sources: [],
        safety,
        mode: 'FIXED' as const,
      },
      reason,
    });
    const decision = checkQuestion(text, audience);
    if (decision.safety !== 'OK') {
      return fixed(decision.safety, decision.text!, decision.reason);
    }
    // The number of earlier questions varies repeated replies ("Hi again!",
    // another joke).
    const friendly = smallTalk(
      text,
      audience,
      earlier.filter((m) => m.role === 'user').length,
    );
    if (friendly) return fixed('OK', friendly, 'small-talk');
    const noSource =
      audience === 'patient'
        ? FIXED_TEXT.noSourcePatient
        : FIXED_TEXT.noSourceClinician;
    let result;
    try {
      result = await this.ai.chatAnswer({
        question: text,
        audience,
        language,
        // The last three questions and answers, for follow-ups.
        history: earlier.slice(-6).map((m) => ({ role: m.role, text: m.text })),
      });
    } catch (err) {
      if (err instanceof AiServiceError && err.status === 409) {
        throw new ConflictException({
          code: 'LANGUAGE_NOT_AVAILABLE',
          message: 'There is no verified content in this language yet.',
        });
      }
      throw new ServiceUnavailableException({
        code: 'CHAT_UNAVAILABLE',
        message: 'The assistant is not available right now. Please try later.',
      });
    }
    if (!result.matched) return fixed('NO_SOURCE', noSource, 'no-match');
    const check = checkAnswer(result.text, result.sources.length);
```

9.2 The safety rules

They are plain pattern checks in `src/services/chatbot/chat-safety.ts`, so they behave the same every time and can be tested:

3-application-logic/backend/src/services/chatbot/chat-safety.ts lines 52–87

```ts
export function checkQuestion(
  text: string,
  audience: ChatAudience,
): SafetyDecision {
  if (SELF_HARM.test(text)) {
    return {
      safety: 'URGENT_CARE',
      reason: 'self-harm',
      text: FIXED_TEXT.urgentSelfHarm,
    };
  }
  if (EMERGENCY.test(text)) {
    return {
      safety: 'URGENT_CARE',
      reason: 'emergency',
      text: FIXED_TEXT.urgentMedical,
    };
  }
  if (MEDICINES.test(text)) {
    return {
      safety: 'DECLINED',
      reason: 'medicines',
      text: FIXED_TEXT.medicines,
    };
  }
  // Clinicians may ask what a value means in general; patients are never
  // told what their own results mean (their clinician explains them).
  if (audience === 'patient' && OWN_RESULTS.test(text)) {
    return {
      safety: 'DECLINED',
      reason: 'own-results',
      text: FIXED_TEXT.ownResults,
    };
  }
  return { safety: 'OK', reason: 'ok' };
}
```

9.3 Small talk

There are 18 kinds of casual message, each with a reply for patients and one for clinicians:

3-application-logic/backend/src/services/chatbot/chat-safety.ts lines 141–170

```
    name: 'how-are-you',
    pattern: whole(
      String.raw`how (?:are|r) (?:you|u|things)(?: (?:doing|today|keeping|going|this morning|tonight))*|how(?:'s| is) it going|how(?:'s| is) your day|how do you do|are you (?:ok(?:ay)?|well|fine|alright|good)|what'?s up|wh?assup|sup`,
    ),
    replies: {
      patient: [
        'I am doing well, thank you for asking! I am here to help with questions about ' +
          HELP_TOPICS_PATIENT +
          '. How can I help you today?',
      ],
      clinician: ['Doing well, thank you. What would you like to look up?'],
    },
  },
  {
    name: 'greeting',
    pattern: whole(
      String.raw`hi+|hello+|hey+|hallo|hiya|howdy|yo|greetings|good (?:morning|afternoon|evening|day)|morning|evening|hi there|hello there|hello hello`,
    ),
    replies: {
      patient: [
        'Hello! I can answer questions about ' +
          HELP_TOPICS_PATIENT +
          '. What would you like to know?',
        'Hi again! What else would you like to know? You can ask about ' +
          HELP_TOPICS_PATIENT +
          '.',
      ],
      clinician: [
        'Hello. I can look up reference information: PI-RADS categories, ISUP grade groups, PSA density and free PSA, DRE findings, and how to read the AI report. What do you need?',
      ],
```

To add a phrase, add it to the `pattern` of the right intent, or add a new block to the `INTENTS` list in `chat-safety.ts`. Then add a line to the table in `chat-safety.spec.ts` and run:

```
cd 3-application-logic\backend
node node_modules\jest\bin\jest.js src/services/chatbot
```

Only a message that is *nothing but* small talk counts: "hello, is my PSA bad?" still gets the safety reply.

9.4 The knowledge base

The assistant can only say what is in two files in `4-ai-intelligence-layer/knowledge-base/en/` : - `patient-learn.json` : the app's Learn articles, word for word; - `clinician-reference.json` : reference cards (PI-RADS, grade groups, PSA density, DRE findings, reading the AI report).

4-ai-intelligence-layer/knowledge-base/en/patient-learn.json lines 1–26

```
{
  "audience": "patient",
  "language": "en",
  "reviewStatus": "Draft for review by a qualified clinician before use with patients (research prototype content).",
  "articles": [
    {
      "id": "prostate",
      "title": "What is the prostate?",
      "summary": "A small gland that helps make semen, and why it can cause problems as men get older.",
      "minutes": 2,
      "sections": [
        {
          "heading": "Where it is and what it does",
          "body": "The prostate is a small gland, about the size of a walnut, found only in men. It sits below the bladder and surrounds the tube that carries urine out of the body (the urethra). It helps to make the fluid that carries sperm (semen)."
        },
        {
          "heading": "Changes with age",
          "body": "The prostate often gets bigger as men get older. This is common and is usually not cancer. A bigger prostate can press on the urethra and cause changes in passing urine, such as needing to go more often, especially at night, or a weaker flow."
        },
        {
          "heading": "Prostate cancer",
          "body": "Prostate cancer happens when cells in the prostate grow out of control. Some prostate cancers grow very slowly and may never cause harm; others can grow and spread. Finding out which kind a man has needs tests and a clinician's assessment."
        }
      ],
      "sources": [
        {
```

Every section of every article is one **passage** the assistant can use. Passages are found in two steps (ADR-013):

1. **Keywords (BM25) decide whether the question is covered.** A passage must share enough of the question's words (a score of 2.0 or more). This needs no model, and every match can be explained by the words it shares with the question.
2. **Meaning (MedCPT) chooses the article.** A biomedical model from the US National Library of Medicine turns the question and each passage into 768 numbers, and the article with the passage closest in meaning comes first. If even the closest passage is far from the question (a score under 52.0), the answer is "no reviewed information". - "Are African men more likely to get it?" now gets the article on who is at higher risk. The keywords alone preferred the one about the gland. - "What is the treatment for malaria?" is refused. It only shares the word "treatment" with a screening passage.
3. **Keywords choose the section inside that article.** The model scores the sections of one article almost the same, so the keywords decide there: "What does PI-RADS 4 mean?" gets the five PI-RADS categories.

4-ai-intelligence-layer/ai-services/app/chat/retrieve.py lines 65–100

```python
class Retriever:
    """BM25 over the passages one audience may see."""

    K1 = 1.5
    B = 0.75

    def __init__(self, kb: KnowledgeBase, audience: Audience, language: str) -> None:
        # Clinicians may also be given the plain patient information.
        allowed = {"patient"} if audience == "patient" else {"patient", "clinician"}
        self.passages = [p for p in kb.passages if p.audience in allowed and p.language == language]
        # The title and heading count twice: they say what the passage is about.
        self._docs = [
            terms(f"{p.title} {p.heading} {p.title} {p.heading} {p.body}") for p in self.passages
        ]
        self._freq = [Counter(d) for d in self._docs]
        n = len(self._docs)
        self._avg_len = sum(len(d) for d in self._docs) / n if n else 0.0
        df: Counter[str] = Counter()
        for d in self._docs:
            df.update(set(d))
        self.idf = {t: math.log(1 + (n - c + 0.5) / (c + 0.5)) for t, c in df.items()}

    def search(self, question: str, top_k: int = 5) -> list[Hit]:
        query = query_terms(question)
        hits: list[Hit] = []
        for passage, doc, freq in zip(self.passages, self._docs, self._freq, strict=True):
            score = 0.0
            for t in query:
                f = freq.get(t, 0)
                if f:
                    norm = self.K1 * (1 - self.B + self.B * len(doc) / self._avg_len)
                    score += self.idf[t] * f * (self.K1 + 1) / (f + norm)
            if score > 0:
                hits.append(Hit(passage, score))
        hits.sort(key=lambda h: h.score, reverse=True)
        return hits[:top_k]
```

4-ai-intelligence-layer/ai-services/app/chat/answer.py lines 132–194

```python
def _rank(
    retriever: Retriever, question: str, text: str, meaning: MeaningScorer | None
) -> tuple[list[Hit], list[Hit]] | None:
    """The passages in order of relevance, and the ones to quote; None when the
    knowledge base does not cover the question. `text` is the question, or a
    short follow-up read together with the previous question."""
    if meaning is None:
        hits = retriever.search(text)
        if not hits or hits[0].score < MIN_SCORE:
            return None
        # Whole passages are quoted: each knowledge base section is written to
        # stand on its own, so a quote never loses the sentence it depends on.
        quoted = [hits[0]]
        if (
            len(hits) > 1
            and hits[1].score >= SECOND_PASSAGE_RATIO * hits[0].score
            and len(hits[0].passage.body) + len(hits[1].passage.body) <= MAX_CHARS
        ):
            quoted.append(hits[1])
        return hits, quoted

    # Meaning search: the keywords decide whether the question is covered.
    # MedCPT tells articles apart well, but scores the sections of one article
    # within a point or two of each other, where the keywords choose better
    # ("What does PI-RADS 4 mean?" is answered by the five categories). So the
    # articles come in order of their closest passage, and inside an article
    # the passages keep their keyword order (candidates are in BM25 order, and
    # the sort is stable).
    hits = retriever.search(text, top_k=len(retriever.passages))
    candidates = [h for h in hits if h.score >= MIN_SCORE]
    if not candidates:
        return None
    closeness = meaning.scores(text, [h.passage for h in candidates])
    article_best: dict[str, float] = {}
    for hit, close in zip(candidates, closeness, strict=True):
        article = hit.passage.article_id
        article_best[article] = max(article_best.get(article, close), close)
    ordered = sorted(
        zip(candidates, closeness, strict=True),
        key=lambda pair: -article_best[pair[0].passage.article_id],
    )
    best = ordered[0][0]
    best_closeness = article_best[best.passage.article_id]
    if best_closeness < MEANING_FLOOR:
        return None
    # A short question read with the previous one must itself be close to the
    # article: "Does it hurt?" after the DRE is, "What is the treatment for
    # malaria?" after the DRE is not (it is a new topic, and not covered).
    if text != question:
        same_article = [
            h.passage for h, _ in ordered if h.passage.article_id == best.passage.article_id
        ]
        if max(meaning.scores(question, same_article)) < MEANING_FLOOR:
            return None
    quoted = [best]
    if (
        len(ordered) > 1
        and ordered[1][1] >= best_closeness - MEANING_SECOND_GAP
        and len(best.passage.body) + len(ordered[1][0].passage.body) <= MAX_CHARS
    ):
        quoted.append(ordered[1][0])
    ranked = [h for h, _ in ordered] + [h for h in hits if h.score < MIN_SCORE]
    return ranked, quoted
```

The model runs inside the AI service, without the internet. It is a 12-layer BERT encoder written with numpy, so it needs no PyTorch:

4-ai-intelligence-layer/ai-services/app/chat/embeddings.py lines 171–203

```
    def _encode(self, ids: Any, mask: Any, types: Any) -> Any:
        np = self._np
        batch, length = ids.shape
        h = (
            self.w["embeddings.word_embeddings.weight"][ids]
            + self.w["embeddings.position_embeddings.weight"][:length]
            + self.w["embeddings.token_type_embeddings.weight"][types]
        )
        h = self._norm(h, "embeddings.LayerNorm")
        size = h.shape[-1] // self.heads
        # Padding gets no attention.
        blocked = ((1.0 - mask[:, None, None, :]) * np.finfo(np.float32).min).astype(np.float32)

        def heads(x: Any) -> Any:
            return x.reshape(batch, length, self.heads, size).transpose(0, 2, 1, 3)

        for i in range(self.layers):
            p = f"encoder.layer.{i}"
            q = heads(self._linear(h, f"{p}.attention.self.query"))
            k = heads(self._linear(h, f"{p}.attention.self.key"))
            v = heads(self._linear(h, f"{p}.attention.self.value"))
            scores = q @ k.transpose(0, 1, 3, 2) / math.sqrt(size) + blocked
            scores = np.exp(scores - scores.max(-1, keepdims=True))
            probs = scores / scores.sum(-1, keepdims=True)
            context = (probs @ v).transpose(0, 2, 1, 3).reshape(batch, length, -1)
            h = self._norm(
                self._linear(context, f"{p}.attention.output.dense") + h,
                f"{p}.attention.output.LayerNorm",
            )
            inner = self._linear(h, f"{p}.intermediate.dense")
            inner = 0.5 * inner * (1.0 + _erf(inner / math.sqrt(2.0)))
            h = self._norm(self._linear(inner, f"{p}.output.dense") + h, f"{p}.output.LayerNorm")
        return h
```

`answer.py` then decides: no passage left means "no reviewed information"; with Claude, Claude writes from the three best passages; otherwise the best passages are quoted.

4-ai-intelligence-layer/ai-services/app/chat/answer.py lines 197–251

```python
def answer(
    kb: KnowledgeBase,
    request: ChatAnswerRequest,
    writer: ClaudeWriter | None = None,
    meaning: MeaningScorer | None = None,
) -> ChatAnswerResult:
    if request.language not in kb.languages():
        raise LanguageNotAvailableError(request.language)
    audience: Audience = request.audience
    retriever = Retriever(kb, audience, request.language)
    info = KnowledgeBaseInfo(version=kb.version, reviewStatus=kb.review_status)
    unmatched = ChatAnswerResult(
        matched=False, text=None, passages=[], sources=[], knowledgeBase=info
    )
    text = _search_text(request)
    try:
        ranking = _rank(retriever, request.question, text, meaning)
    except MeaningBusyError:
        # Every slot of the meaning search is taken (heavy load): keywords
        # only, as before ADR-013, so the answer still comes within 2 s.
        ranking = _rank(retriever, request.question, text, None)
    if ranking is None:
        return unmatched
    hits, quoted = ranking

    if writer is not None:
        given = hits[:PASSAGES_FOR_CLAUDE]
        written = writer.write(
            request.question,
            audience,
            [h.passage for h in given],
            [Turn(role=t.role, text=t.text) for t in request.history],
        )
        if written is not None and not written.covered:
            return unmatched.model_copy(update={"mode": "GENERATED", "model": written.model})
        if written is not None and written.cited:
            used = [given[i] for i in written.cited]
            return ChatAnswerResult(
                mode="GENERATED",
                matched=True,
                text=written.text,
                passages=[_passage_out(h) for h in used],
                sources=_sources([h.passage for h in used]),
                knowledgeBase=_info(kb, [h.passage for h in used]),
                model=written.model,
            )
        # No usable answer from Claude: quote the passages instead.

    return ChatAnswerResult(
        matched=True,
        text=" ".join(h.passage.body for h in quoted),
        passages=[_passage_out(h) for h in quoted],
        sources=_sources([h.passage for h in quoted]),
        knowledgeBase=_info(kb, [h.passage for h in quoted]),
    )
```

Setting up the meaning search (once). In `4-ai-intelligence-layer\ai-services`, run `.venv\Scripts\python -m app.chat.embeddings download`. - It fetches the two MedCPT encoders (about 880 MB) into `models\`. Git ignores that folder. - It refuses any file whose checksum differs from the pinned one. - Then restart the AI service. A few seconds later, the health check says `chat_retrieval: keywords+meaning (MedCPT)`. - Without the models, the assistant answers with keywords only, as it did before. `dev-up.ps1` reminds you to download them.

Under heavy load the meaning search handles 6 questions at a time on this laptop (`CHAT_MEANING_SLOTS`). A question that waits more than half a second for its turn (`CHAT_MEANING_WAIT_MS`) is answered with keywords only, so answers stay within 2 seconds. The health check counts those questions (`chat_meaning_busy`). With 100 clinicians asking at once, none needed it (performance.md).

To change what the assistant knows:

1. Edit the JSON file. Keep each section short and complete in itself, and give every article a source.
2. If you changed `patient-learn.json` , copy it over `1-presentation-layer/mobile-app/assets/education/en/articles.json` . A test fails if the two differ.
3. Restart the AI service. It reads the files when it starts, and works out the new passages' meaning vectors in the background (about 12 seconds).
4. Run its tests: `.venv\Scripts\python -m pytest -q` .
5. Check the search on the question sets: `.venv\Scripts\python -m app.chat.evaluate` . It shows which questions changed their answer, and the meaning scores next to the 52.0 floor. Add new kinds of questions to `app/chat/eval_sets.py` .

The content says "*Draft for review by a qualified clinician*" until a clinician signs it off. Record a sign-off by changing `reviewStatus` in the file and noting the reviewer and date in the development log.

9.5 Claude (optional, costs money)

Claude is **off** on this PC: the health check says `chat_writer: quotes` . To switch it on:

1. Create an API key at console.anthropic.com (Settings → API keys), and add a little credit under Billing. A spending limit there caps the cost.
2. Open the root `.env` and put the key after `ANTHROPIC_API_KEY=` , with no spaces or quotes. Never put it in a chat, an email or the code.
3. Restart the services: `6-infrastructure\scripts\dev-down.ps1` , then `6-infrastructure\scripts\dev-up.ps1` .
4. Open http://127.0.0.1:8000/v1/health. It should say `chat_writer: claude:claude-haiku-4-5-20251001` .
5. Make one real call (costs a fraction of a cent). In `4-ai-intelligence-layer\ai-services` , run `$env:ANTHROPIC_API_KEY_LIVE_TEST = '1'; .venv\Scripts\python -m pytest tests\test_chat_claude.py -k live -q` . One test should pass.
6. In the app, ask the assistant "What does a PSA test measure?". The answer should carry the label "*Written by AI (Claude) from the sources below*". If it is quoted word for word instead, the call failed: check the key, the credit and the internet connection.

To switch Claude off again, empty `ANTHROPIC_API_KEY=` and restart. Another model can be set with `CHAT_LLM_MODEL=` in `.env` . The request works with every current Claude model, but measure the speed again (ADR-010).

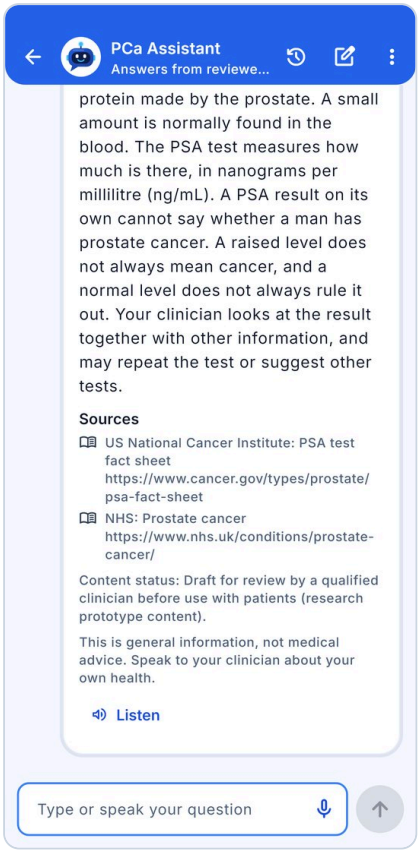
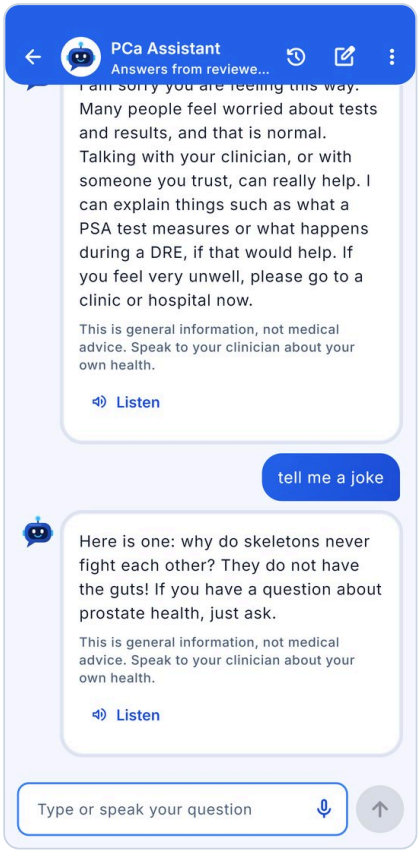
What Claude is told:

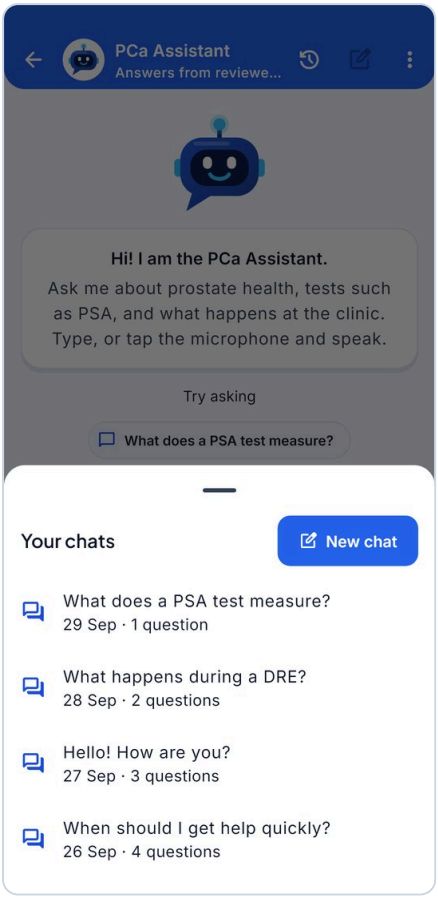
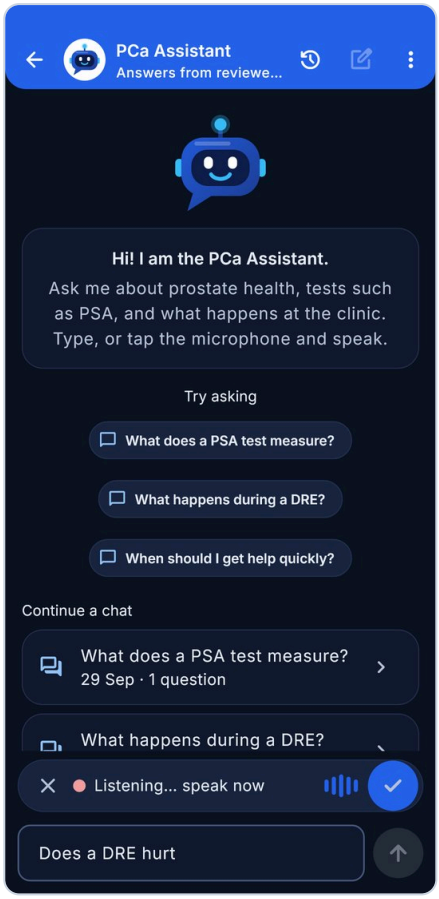
4-ai-intelligence-layer/ai-services/app/chat/generate.py lines 51–65

```python
SYSTEM: dict[Audience, str] = {
    "patient": (
        "You are the PCa mHealth assistant, answering questions from patients in Zambia about "
        "the prostate, PSA tests, the digital rectal exam, screening and when to get help. "
        "Write warmly and simply: short sentences, everyday words, a reading age of about 12, "
        "at most about 120 words. When it helps, suggest a question they could ask their "
        "clinician." + _COMMON_RULES
    ),
    "clinician": (
        "You are the PCa mHealth reference assistant for clinicians in Zambia. Answer concisely "
        "and precisely, at most about 150 words, and keep all definitions and numbers exactly as "
        "in the passages. The answer supports but does not replace clinical judgement or the "
        "local protocol." + _COMMON_RULES
    ),
}
```

- Claude only receives the three best passages, the last three questions and answers, and the question. Email addresses, NRC numbers and phone numbers are removed first.
- It must answer in a fixed JSON form (`covered` , `answer` , `citations`), which the API enforces (structured output). Only passages it cites become the sources.
- Limits: 8 seconds, no retries, at most `CHAT_LLM_DAILY_LIMIT` (2,000) answers a day. On any problem the passages are quoted instead, so the chat works without Claude.
- Answers written by Claude are labelled "*Written by AI (Claude) from the sources below*".
- **Before real patients:** questions would be sent to Anthropic in the United States. That needs a data-protection review first (ADR-010).

9.6 What the person sees

Empty chat	An answer with its source	Casual chat
		

Past chats	Speaking a question
	

These screens are drawn by the app's own code with synthetic data. The answer is the AI service's real answer to that question (quoted, because Claude is off), and the casual replies are the backend's real replies. In the recording picture a stand-in plays the part of the phone's speech service.

9.7 Testing the chatbot

What	Command	Where
Safety rules and small talk (89 tests)	`node node_modules\jest\bin\jest.js src/services/chatbot`	`3-application-logic\backend`
The whole chat through the API and MongoDB (11 tests)	`node node_modules\jest\bin\jest.js --config ./test/jest-db.json --runInBand test/db/chatbot.int-spec.ts`	`3-application-logic\backend`
Search quality (keywords and meaning), and Claude with a stand-in	`.venv\Scripts\python -m pytest -q`	`4-ai-intelligence-layer\ai-services`
The search measured on the question sets	`.venv\Scripts\python -m app.chat.evaluate`	`4-ai-intelligence-layer\ai-services`
The chat screens	`flutter test test/features/chat`	`1-presentation-layer\mobile-app`

10. The phone app

10.1 How it is organised

`1-presentation-layer/mobile-app/lib` :

Folder	What is inside
`main.dart`	the first code that runs
`app/`	the routes (which screen is at which address, and who may open it) and the theme (colours, type, shapes)
`core/`	shared machinery: `network` (talking to the API), `db` (the encrypted database on the phone), `sync` (sending offline work later), `storage` (the session tokens), `uploads`
`features/`	one folder per feature: `auth` , `home` , `patients` , `clinical_server` , `patient` , `chat` , `sync` , `settings`
`shared/`	widgets used everywhere (the header, tiles, the assistant's face) and `audio` (read-aloud)

`main.dart` opens the phone's encrypted database and the saved preferences, then starts the app:

1-presentation-layer/mobile-app/lib/main.dart — lines 14–28

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  _registerFontLicences();
  final database = await openAppDatabase();
  final preferences = await SharedPreferences.getInstance();
  // Push notifications only when built with the Firebase settings (ADR-014).
  final push = await startFirebasePush();
  runApp(
    ProviderScope(
      overrides: [
        appDatabaseProvider.overrideWithValue(database),
        preferencesProvider.overrideWithValue(preferences),
        pushMessagingProvider.overrideWithValue(push),
      ],
      child: const PcaApp(),
```

The only setting built into the app is the server's address. No secret is ever compiled in:

1-presentation-layer/mobile-app/lib/core/config/app_env.dart lines 1–18

```dart
/// Build-time configuration. No secrets are ever compiled into the app: only
/// the API address (`--dart-define=API_BASE_URL=...`) and, for push
/// notifications, the public identifiers of the owner's Firebase project.
abstract final class AppEnv {
  /// Defaults to the host machine as seen from the Android emulator.
  static const apiBaseUrl = String.fromEnvironment(
    'API_BASE_URL',
    defaultValue: 'http://10.0.2.2:3000',
  );

  /// Where administrators manage the system (a separate web app).
  static const adminPortalUrl = String.fromEnvironment(
    'ADMIN_PORTAL_URL',
    defaultValue: 'http://localhost:5173',
  );

  static String get apiRoot =>
      '${apiBaseUrl.replaceAll(RegExp(r'/+$'), '')}/api/v1';
```

10.2 Running the app

On the emulator (the backend must be running):

```
flutter emulators --launch Galaxy_S9_Plus_API_29
cd 1-presentation-layer\mobile-app
flutter pub get      # only after pubspec.yaml changed
flutter run -d emulator-5554 --dart-define=API_BASE_URL=http://10.0.2.2:3000
```

`10.0.2.2` is how the emulator reaches this PC. While `flutter run` is open, press **r** to reload after a code change, **R** to restart the app, **q** to quit. In Android Studio, the run configuration **App - S9+ emulator** does the same.

On a real Android phone by USB:

1. On the phone: Settings › Developer options › USB debugging on. Plug it in and tap **Allow**.
2. `powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\phone-usb.ps1` (it lets the phone reach the PC's port 3000).
3. `flutter run --dart-define=API_BASE_URL=http://localhost:3000`, or the Android Studio configuration **App - USB phone**.

To make an installable file (APK):

```
flutter build apk --release --dart-define=API_BASE_URL=https://your-server.example
```

The file appears in `build\app\outputs\flutter-apk\app-release.apk`. A release build for the public also needs a signing key and an HTTPS server address.

10.3 How the app is put together

- **State** is managed with Riverpod: a *controller* holds the state of one screen and the screen redraws when it changes. The chat controller is a typical example:

1-presentation-layer/mobile-app/lib/features/chat/application/chat_controller.dart lines 82–117

```dart
class ChatController extends Notifier<ChatState> {
  @override
  ChatState build() => const ChatState();

  Future<void> ask(String text) async {
    final question = text.trim();
    if (question.isEmpty || state.sending || state.loading) return;
    state = state.copyWith(sending: true, clearError: true);
    final api = ref.read(chatApiProvider);
    try {
      final id = state.conversationId ?? await api.start();
      final (asked, answer) = await api.ask(id, question);
      state = state.copyWith(
        conversationId: id,
        messages: [...state.messages, asked, answer],
        sending: false,
      );
      ref.invalidate(chatHistoryProvider);
    } catch (e) {
      state = state.copyWith(sending: false, error: chatErrorText(e));
    }
  }

  /// Opens an earlier conversation so it can be read and continued.
  Future<void> open(String conversationId) async {
    if (state.sending || conversationId == state.conversationId) return;
    state = ChatState(conversationId: conversationId, loading: true);
    try {
      final messages = await ref.read(chatApiProvider).messages(conversationId);
      if (state.conversationId != conversationId) return;
      state = ChatState(conversationId: conversationId, messages: messages);
    } catch (e) {
      if (state.conversationId != conversationId) return;
      state = ChatState(error: chatErrorText(e));
    }
  }
```

- **Talking to the server:** `core/network/api_client.dart` adds the session token to every request, renews it when it has expired, and turns server errors into plain messages.
- **Offline:** clinicians' work is saved first in an encrypted database on the phone (Drift with SQLCipher) and sent when the connection returns (`core/sync`). The server re-checks every change with the same rules as online.
- **Session tokens** are kept in the phone's secure storage, never in plain files.
- **Permissions:** the app only asks for the internet; the microphone, when it is first tapped; and, for patients after signing in, to show notifications (Android 13 and later, and only when the app is built with the Firebase settings, section 18).
- **Push notifications** (`core/push`): after a patient signs in, the phone is registered with the server, and it is removed again at sign-out (section 18).

10.4 Voice and reading aloud

Both use the phone's own speech services, behind small interfaces, so tests can replace them with stand-ins.

Voice (`features/chat/application/voice_input.dart`): the phone turns speech into text, and the app only receives the words.

1-presentation-layer/mobile-app/lib/features/chat/application/voice_input.dart lines 74–102

```dart
Future<void> listen({
  required void Function(String words) onWords,
  required void Function(double level) onLevel,
  required void Function() onDone,
}) async {
  _onDone = onDone;
  // The answers are in English: use the phone's English locale, or
  // British English when the phone is set to another language.
  final system = await _speech.systemLocale();
  final locale = system != null && system.localeId.startsWith('en')
      ? system.localeId
      : 'en_GB';
  await _speech.listen(
    onResult: (result) {
      onWords(result.recognizedWords);
      if (result.finalResult) _finish();
    },
    // Android reports roughly -2 to 10 dB.
    onSoundLevelChange: (level) => onLevel(((level + 2) / 12).clamp(0, 1)),
    listenOptions: stt.SpeechListenOptions(
      listenFor: const Duration(seconds: 60),
      pauseFor: const Duration(seconds: 3),
      localeId: locale,
      partialResults: true,
      cancelOnError: true,
      listenMode: stt.ListenMode.dictation,
    ),
  );
}
```

Read-aloud (`shared/audio/read_aloud.dart`): an article is read part by part, so the screen can highlight the part being read.

1-presentation-layer/mobile-app/lib/shared/audio/read_aloud.dart lines 110–139

```dart
class ReadAloudController extends Notifier<ReadAloudState> {
  int _run = 0;
  List<String> _parts = const [];

  ReadAloud get _voice => ref.read(readAloudProvider);

  @override
  ReadAloudState build() => const ReadAloudState();

  Future<void> play(String id, List<String> parts, {int from = 0}) async {
    final run = ++_run;
    await _voice.stop();
    _parts = parts;
    if (!ref.mounted) return;
    await _voice.setRate(state.slow ? slowReadingRate : normalReadingRate);
    for (var i = from; i < parts.length; i++) {
      // Stopped, or another text started, or the app is closing.
      if (!ref.mounted || run != _run) return;
      state = ReadAloudState(
        id: id,
        part: i,
        total: parts.length,
        slow: state.slow,
      );
      await _voice.speak(parts[i]);
    }
    if (ref.mounted && run == _run) state = ReadAloudState(slow: state.slow);
  }

  Future<void> pause() async {
```

The assistant's face is not a picture file. It is drawn by code, so it is sharp at every size:

1-presentation-layer/mobile-app/lib/shared/widgets/assistant_avatar.dart lines 110–153

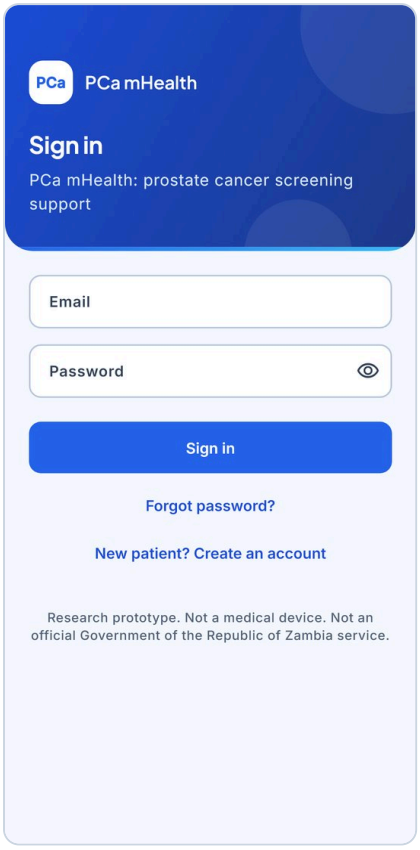
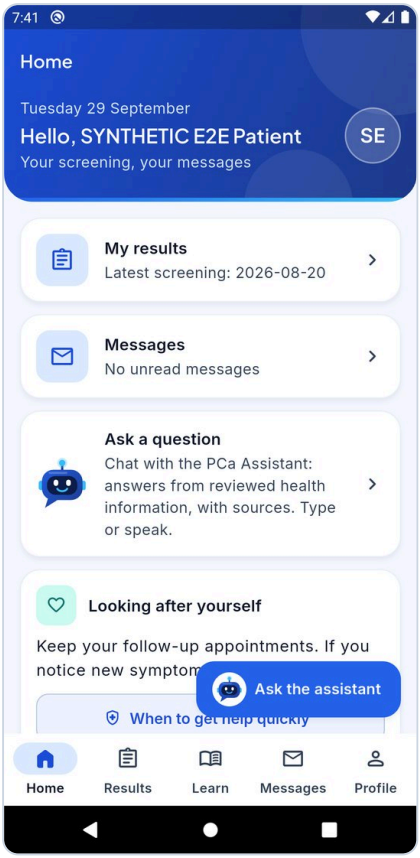
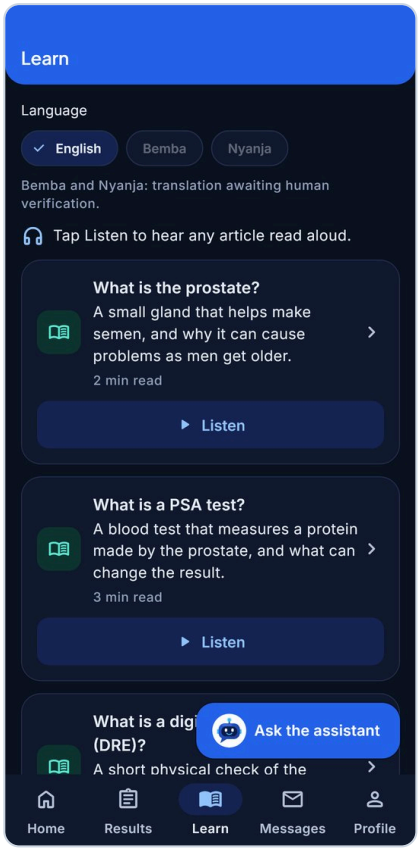
```dart
  void paint(Canvas canvas, Size size) {
    final u = size.shortestSide / 100;
    Offset o(double x, double y) => Offset(x * u, y * u);
    Rect box(double l, double t, double r, double b) =>
        Rect.fromLTRB(l * u, t * u, r * u, b * u);

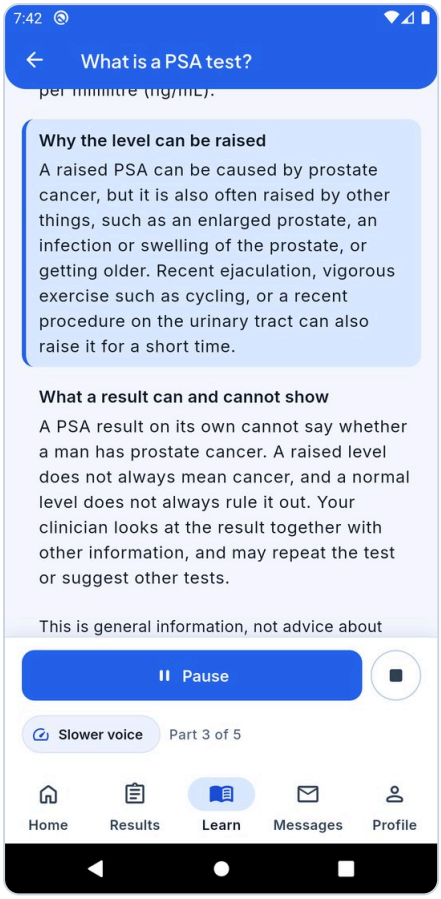
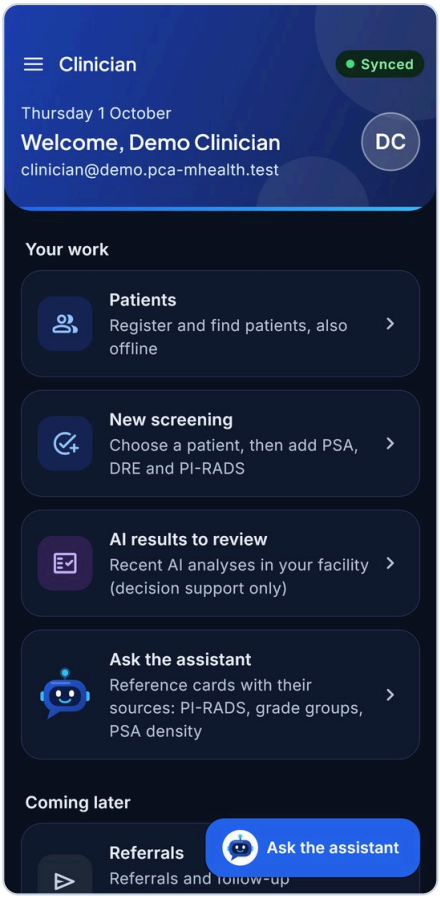
    // Antenna with a glowing awareness-blue tip.
    canvas.drawLine(
      o(50, 25),
      o(50, 13),
      Paint()
        ..color = _head.last
        ..strokeWidth = 4.5 * u
        ..strokeCap = StrokeCap.round,
    );
    canvas.drawCircle(
      o(50, 10),
      9.5 * u,
      Paint()..color = _accent.withValues(alpha: 0.3),
    );
    canvas.drawCircle(o(50, 10), 6 * u, Paint()..color = _accent);

    // Ears.
    final ear = Paint()..color = _accent;
    for (final (l, r) in [(5.0, 13.0), (87.0, 95.0)]) {
      canvas.drawRRect(
        RRect.fromRectAndRadius(box(l, 44, r, 62), Radius.circular(4 * u)),
        ear,
      );
    }

    // Head and speech-bubble tail, one shape with a blue gradient.
    final head = Path()
      ..addRRect(
        RRect.fromRectAndRadius(box(11, 23, 89, 81), Radius.circular(22 * u)),
      );
    final tail = Path()
      ..moveTo(24 * u, 74 * u)
      ..lineTo(17 * u, 96 * u)
      ..lineTo(44 * u, 79 * u)
      ..close();
    canvas.drawPath(
      Path.combine(PathOperation.union, head, tail),
      Paint()
```

10.5 What it looks like

Sign-in	Patient home (emulator)	Learn, dark mode
		

An article being read aloud (emulator)	Clinician home, dark mode
	

10.6 App commands

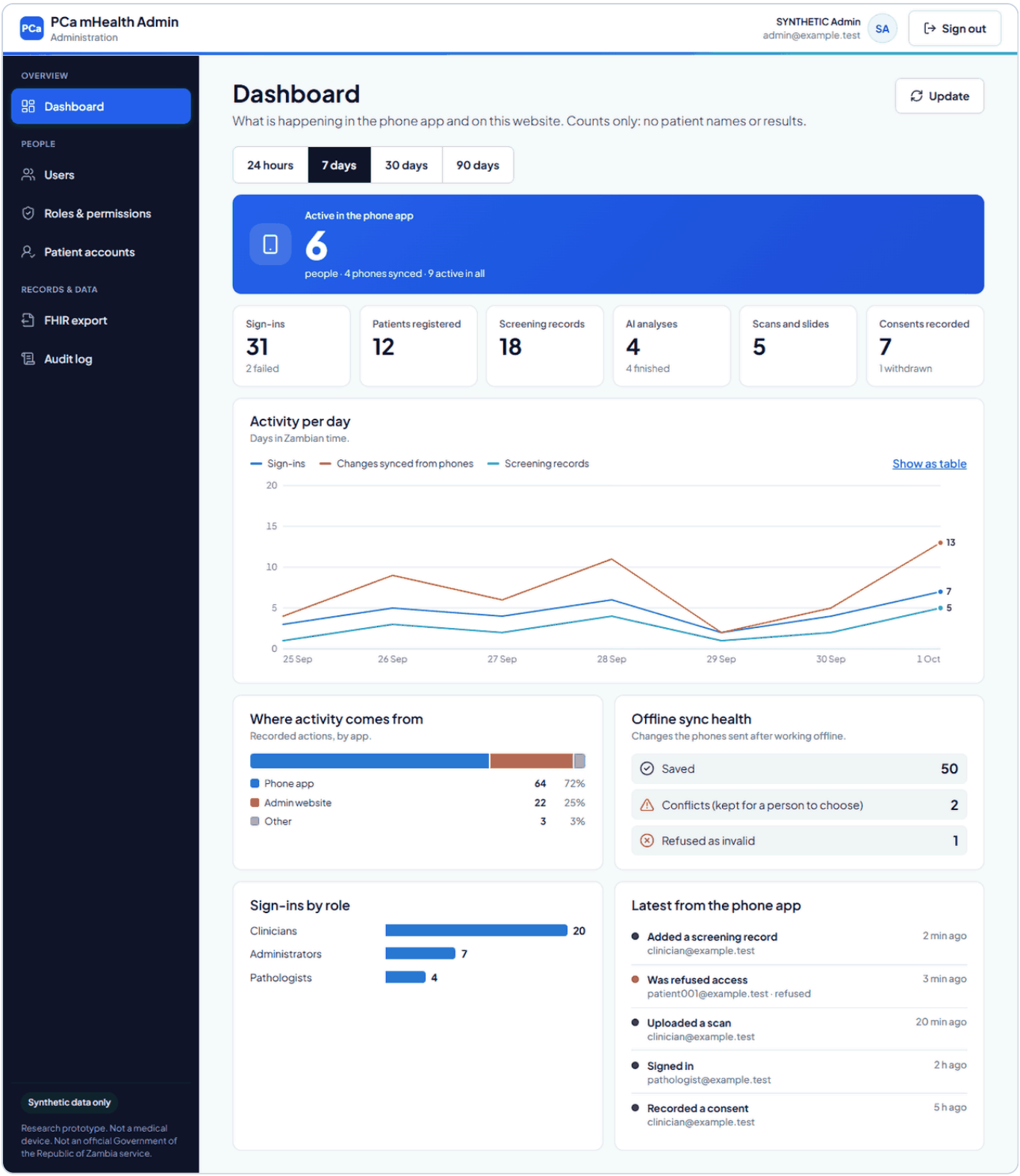
Run these in `1-presentation-layer\mobile-app`:

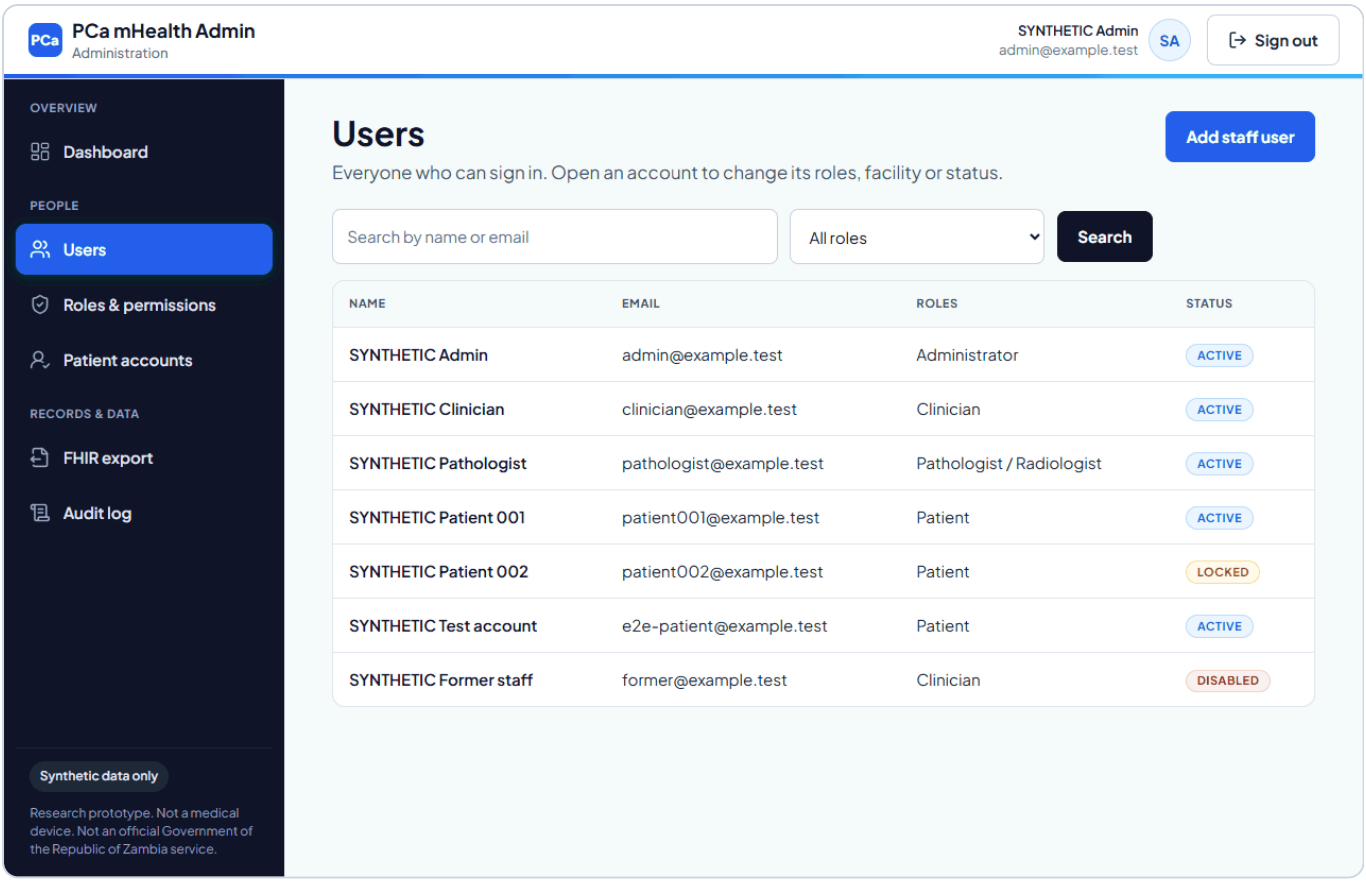
Task	Command
Get packages	`flutter pub get`
Check the code	`flutter analyze`
Format	`dart format lib test`
All tests	`flutter test`
One test file	`flutter test test/features/chat/chat_test.dart`
Build a debug APK	`flutter build apk --debug`
List devices	`flutter devices`

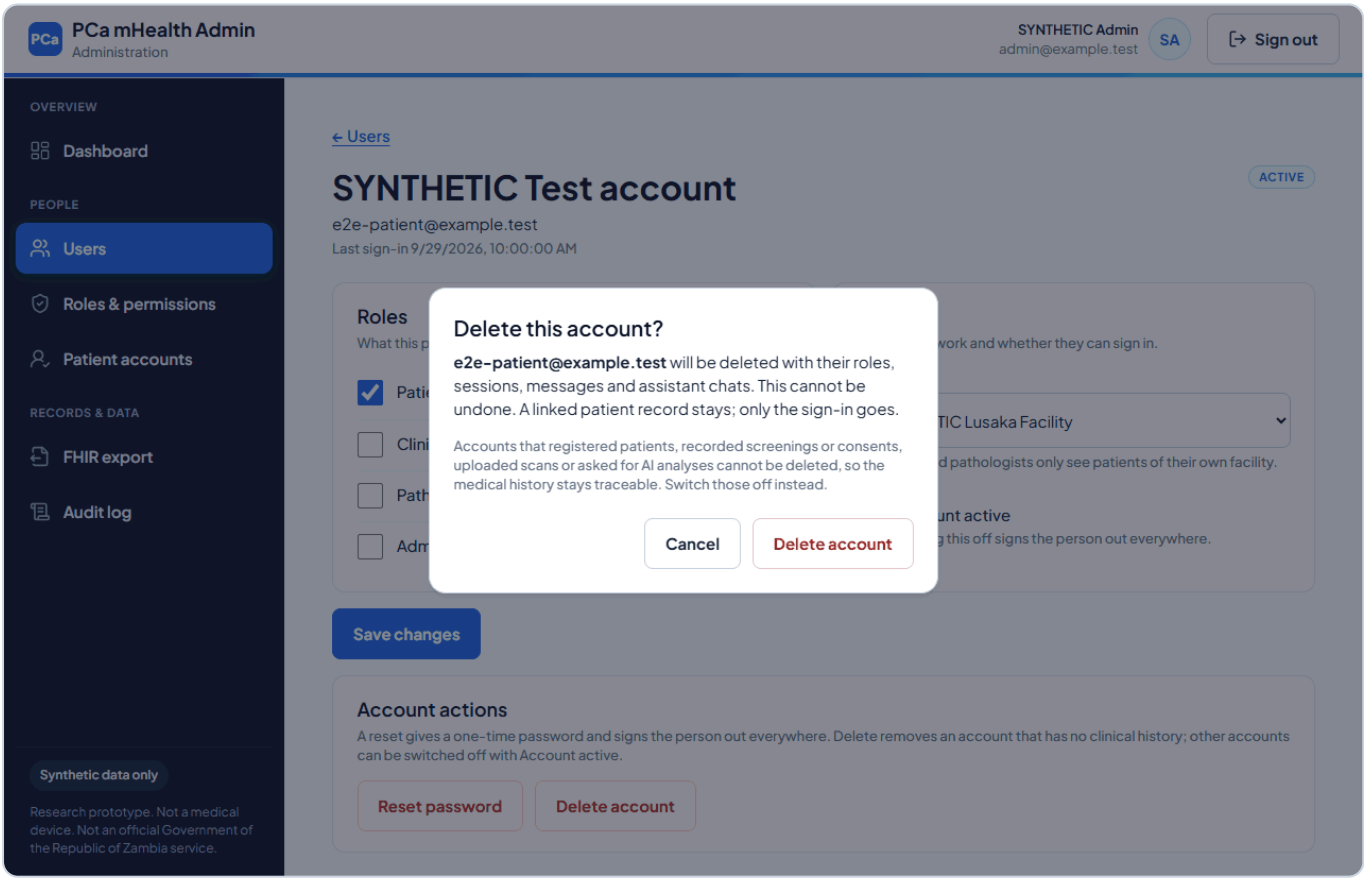
11. The admin website

`1-presentation-layer/admin-panel-web`, a React application. Only administrators can sign in.

Page	What it is for
Dashboard	what is happening in the phone app: sign-ins, records, sync health (counts only, no patient names)
Users	every account: roles, facility, switch off, unlock, reset password, **delete**
Roles & permissions	what each role may do
Patient accounts	link a patient's sign-in to their clinic record
FHIR export	export records for SmartCare Pro
Audit log	who did what, and **Check integrity**


PCa mHealth Admin
Administration
SYNTHETIC Admin
admin@example.test
SA
Sign out
OVERVIEW
Dashboard
PEOPLE
Users
Roles & permissions
Patient accounts
RECORDS & DATA
FHIR export
Audit log
Synthetic data only
Research prototype. Not a medical device. Not an official Government of the Republic of Zambia service.
Dashboard
What is happening in the phone app and on this website. Counts only: no patient names or results.
Update
24 hours
7 days
30 days
90 days
Active in the phone app
6
people · 4 phones synced · 9 active in all
Sign-ins
31
2 failed
Patients registered
12
Screening records
18
AI analyses
4
4 finished
Scans and slides
5
Consents recorded
7
1 withdrawn
Activity per day
Days in Zambian time.
Sign-ins
Changes synced from phones
Screening records
Show as table
20
15
10
5
0
13
7
5
25 Sep
26 Sep
27 Sep
28 Sep
29 Sep
30 Sep
1 Oct
Where activity comes from
Recorded actions, by app.
Phone app 64 72%
Admin website 22 25%
Other 3 3%
Offline sync health
Changes the phones sent after working offline.
Saved 50
Conflicts (kept for a person to choose) 2
Refused as invalid 1
Sign-ins by role
Clinicians 20
Administrators 7
Pathologists 4
Latest from the phone app
Added a screening record
clinician@example.test
2 min ago
Was refused access
patient001@example.test · refused
3 min ago
Uploaded a scan
clinician@example.test
20 min ago
Signed in
pathologist@example.test
2 h ago
Recorded a consent
clinician@example.test
5 h ago





These pictures show the real website, but the API's answers were replaced with synthetic ones while they were taken: the accounts are not real, and the dashboard's numbers were made up for the picture.

The dialog in that picture is this code in `src/pages/UsersPages.tsx`:

1-presentation-layer/admin-panel-web/src/pages/UsersPages.tsx lines 405–438

```tsx
      {confirmDelete && (
        <Dialog
          title="Delete this account?"
          onClose={() => setConfirmDelete(false)}
          actions={
            <>
              <button onClick={() => setConfirmDelete(false)}>Cancel</button>
              <button
                className="danger"
                disabled={remove.isPending}
                onClick={() => remove.mutate()}
              >
                {remove.isPending ? 'Deleting…' : 'Delete account'}
              </button>
            </>
          }
        >
          <p>
            <strong>{user.email}</strong> will be deleted with their roles,
            sessions, messages and assistant chats. This cannot be undone. A
            linked patient record stays; only the sign-in goes.
          </p>
          <p className="muted">
            Accounts that registered patients, recorded screenings or consents,
            uploaded scans or asked for AI analyses cannot be deleted, so the
            medical history stays traceable. Switch those off instead.
          </p>
        </Dialog>
      )}
      {temp && (
        <TemporaryPasswordDialog
          email={user.email}
          password={temp}
          onClose={() => setTemp(null)}
```

Commands (in `1-presentation-layer\admin-panel-web`):

Task	Command
Run for development	`npm run dev`
Tests	`npm test`
Tests with coverage	`npm run test:cov`
Lint, types	`npm run lint`, `npm run typecheck`
Build the files for a web server	`npm run build` (output in `dist/`)

12. Tests and the quality gate

One script checks a whole part: formatting, lint, types, tests, build and a scan of the packages for known problems.

```
$gate = "6-infrastructure\scripts\quality-gate.ps1"
powershell -ExecutionPolicy Bypass -File $gate backend
powershell -ExecutionPolicy Bypass -File $gate mobile
powershell -ExecutionPolicy Bypass -File $gate admin-web
powershell -ExecutionPolicy Bypass -File $gate ai
# The databases' tests (needs the databases and Docker):
powershell -ExecutionPolicy Bypass -File $gate db
# Every link in the docs works; are the manual's pictures current?
powershell -ExecutionPolicy Bypass -File $gate docs
# No password or key in Git:
powershell -ExecutionPolicy Bypass -File $gate secrets
powershell -ExecutionPolicy Bypass -File $gate all
```

`quality-gate.ps1` runs the real script, `quality-gate.sh`, with the `bash` that comes with Git for Windows. In **Git Bash** you can run that directly: `bash 6-infrastructure/scripts/quality-gate.sh backend`. Do not type `bash …` in PowerShell: on this PC it starts Windows' WSL, not Git Bash.

The quality gate for the backend

```
PS D:\Final Year Project\pca-mhealth> powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 backend
(only the summary lines are shown)
==> backend
    PASS: backend format
    PASS: backend lint
    PASS: backend typecheck
    PASS: prisma validate
Tests:       323 passed, 323 total
    PASS: backend unit tests
Tests:       20 passed, 20 total
    PASS: backend e2e tests
    PASS: openapi document up to date
    PASS: FHIR definitions up to date
    PASS: access matrix up to date and within the review rules
    PASS: backend build
    PASS: backend npm audit
QUALITY GATE PASSED (backend)
```

Where things stood on 1 October 2026:

Part	Tests
Backend unit tests	323
Backend end-to-end tests	20
Backend database tests	238 (all backend tests together: 94.7% of statements, 81.4% of branches)
AI service	96, and 1 skipped (it would call the real Claude); without the MedCPT models, 3 more are skipped
Phone app	290 (line coverage 91.1%)

Part	Tests
Admin website	81 (coverage 87.8%)

The rule the project has followed: no commit unless the gate for the changed part passes. If a test fails, fix the code or, if the behaviour really should change, change the test and say why in the commit.

13. Git and GitHub

- The code is in a **private** GitHub repository, `deedeestephen/final-year-project`, branch `main`.
- The history is one commit per finished piece of work, with a message that says what and why.

```
git status                      # what changed
git diff                        # the changes, line by line
git add -A                      # stage everything
git commit -m "What changed and why"
git push origin main            # send to GitHub
git log --oneline -10           # the last ten commits
git pull origin main            # get changes from GitHub
```

Never commit `.env`, passwords, keys or real patient data. Before committing, run `powershell -ExecutionPolicy Bypass -File 6-infrastructure\scripts\quality-gate.ps1 secrets`.

14. When something goes wrong

What you see	Likely cause	What to do
`dev-up.ps1`: *The .env file is missing*	new folder or new PC	`node 6-infrastructure\scripts\gen-keys.mjs --init-env`, then set the database passwords
`dev-up.ps1`: *Database migration failed*	PostgreSQL is stopped, or `DATABASE_URL` is wrong	`Get-Service postgresql-x64-18`; check the port (5432) and password in `.env`
`dev-up.ps1`: *docker compose failed*	Docker Desktop is not running	open Docker Desktop, wait for "Engine running", run again
Backend window: `EADDRINUSE :::3000`	an old backend still runs	run `dev-down.ps1`, then `dev-up.ps1`
Backend window: MongoDB `Authentication failed`	`MONGO_URL` and the `pca` password differ	see the troubleshooting table in local-databases.md
Admin website: *Email or password is incorrect*	wrong password, or it was changed	use `SEED_DEMO_PASSWORD` from `.env` (not `.env.example`), or `dev-up.ps1 -ResetDemoPasswords`

What you see	Likely cause	What to do
This account is temporarily locked	5 wrong tries	wait 15 minutes, or `dev-up.ps1 -ResetDemoPasswords`
Admin website: *Cannot reach the server*	the backend is not running	check http://localhost:3000/api/v1/health
App: *Cannot reach the server* on the emulator	the backend is not running, or the wrong address	the address must be `http://10.0.2.2:3000` on the emulator
App shows a white screen after the PC restarted	a debug build needs `flutter run`	run `flutter run` again (section 10.2)
Backend window: *FCM_SERVICE_ACCOUNT_FILE: …* and it stops	the push key file's path is wrong, or it is not the service-account key	fix the path in `.env`, or empty it to switch push off (section 18)
No push arrives on the phone	push is off on the server, the app was built without `firebase-app.json`, or notifications are not allowed	check `"push":"on"` at http://localhost:3000/api/v1/health/ready, run the app as in section 18, allow notifications in the phone's settings
Chat: *The assistant is not available right now*	the AI service is not running	start it (section 3.4) and check http://127.0.0.1:8000/v1/health
Chat always answers *No reviewed information*	the question is outside the knowledge base	expected; add content (section 9.4)
Voice: *I did not hear anything* on the emulator	the emulator's microphone is off	emulator › ⋮ › Microphone › *Virtual microphone uses host audio input*, or use a real phone
A test run ends at once with exit code 127 and no output	the PC ran out of memory	close Android Studio or the emulator, run `cd android; .\gradlew --stop`, try again
Docker prints *The "…" variable is not set*	a `.env` value contains `$` without quotes	put the value in single quotes
pgAdmin does not open	it must be started from the Start menu	Start › PostgreSQL 18 › pgAdmin 4

Where the messages are - Backend: its PowerShell window. - AI service: its PowerShell window. - Admin website: the browser's console (press F12). - App: the `flutter run` window. - Docker: `docker compose … logs <service>`. - What happened in the system: the **Audit log** page on the admin website.

15. Before real patients

The prototype runs on synthetic data. These are still needed before any real use, and none of them is a programming task alone:

- **Trained and evaluated AI models.** The mocks stay labelled until then.
- **A clinician's sign-off** of the knowledge base, and **human-verified Bemba and Nyanja** translations.
- **A data-protection review** under Zambia's Data Protection Act (2021) and ethics approval, covering the speech service and, if used, Claude and Firebase push.
- **MongoDB access control switched on**, and a proper server with HTTPS, backups and monitoring. The list is in security-review.md, items R-1 to R-8.

16. Where to read more

Question	Read
How do I test every feature by hand?	how-to-test.md
How do I run the user acceptance test with real users?	uat/README.md (only after ethics approval)
Why was it built this way?	architecture.md and decisions/
What was done, when, and what was found?	development-log.md
Which requirement is met by what, and how is it tested?	requirements-traceability.md
The databases on this PC	local-databases.md and database.md
The phone app in depth	mobile.md
Security	security.md, security-review.md, access-matrix.md
The chatbot's design	chatbot-plan.md, ADR-009, ADR-010, ADR-013
Push notifications	ADR-014, and section 18 above
Swapping the mock AI models for real ones	ai-model-integration-guide.md
Speed and load	performance.md, scalability.md

17. Updating this manual

This text is `docs/operations-manual.md`: change it in any editor. Its pictures, its PDF and its web page are made by a tool in `docs/report/tools`, from this text and from the code, so they can be made again whenever either changes:

```
$report = "docs\report\tools\report.ps1"
# What is out of date:
powershell -ExecutionPolicy Bypass -File $report check
# After changing only the text:
powershell -ExecutionPolicy Bypass -File $report pdf html
# After changing code that a picture shows:
powershell -ExecutionPolicy Bypass -File $report code images pdf html
# Everything (start dev-up.ps1 first):
powershell -ExecutionPolicy Bypass -File $report all
```

- It needs Python, Node and Chrome (or Edge). The first run installs three small Python packages.
- `docs/report/tools/contents.py` lists what each picture shows: which file and lines, which command, which phone screen.
- The quality gate's `docs` target runs `check`. It warns when a code picture shows code that has changed since the picture was taken.
- Every step, what it needs, and what to do when one stops: report/tools/README.md.
- The same tool draws the **diagrams for the final-year report** (use cases, data flow, activity, sequence, state, class, database, API, architecture, timelines) from text files in `docs/report/diagrams`: `report.ps1 diagrams figures`. They are listed, with captions, in report-figures.md.

18. Push notifications (switching them on)

Patients can get a notification on their phone when a screening record is added, a consent is given or withdrawn, or their account is linked to their clinic record (ADR-014). - The same messages are always in the app's **Messages** tab; a push only tells the person sooner. - A push carries only the message's title and text, which never contain results, names or numbers. A locked phone hides even those. - **Push is off on this PC.** It needs a Firebase project, which belongs to your Google account. Firebase Cloud Messaging is free.

Once, in the Firebase console (console.firebase.google.com, signed in with your Google account):

1. **Create a project,** for example *pca-mhealth*. Google Analytics is not needed.
2. **Add an Android app** with the package name `zm.ac.zcas.pca_mhealth`.
3. **Download** `google-services.json` when the console offers it. The app does not use the file, and the Gradle steps can be skipped; you only copy four values from it (step 7).
4. **Project settings › Cloud Messaging:** check that *Firebase Cloud Messaging API (V1)* is **Enabled**.
5. **Project settings › Service accounts › Generate new private key.** A JSON file downloads. - It is a **secret**, like a password: anyone with it can send pushes as your project. - Save it **outside the project folder**, for example `D:\Final Year Project\secrets\pca-mhealth-firebase.json`. - Never e-mail it or put it in Git.

On this PC:

6. Open `.env` and set the full path of that file: `FCM_SERVICE_ACCOUNT_FILE=D:\Final Year Project\secrets\pca-mhealth-firebase.json`.
7. In `1-presentation-layer\mobile-app`, copy `firebase-app.example.json` to `firebase-app.json` (Git ignores it) and fill in the four values from `google-services.json`:

In `firebase-app.json`	**From** `google-services.json`
`FIREBASE_API_KEY`	`client` › `api_key` › `current_key`
`FIREBASE_APP_ID`	`client` › `client_info` › `mobilesdk_app_id`
`FIREBASE_SENDER_ID`	`project_info` › `project_number`
`FIREBASE_PROJECT_ID`	`project_info` › `project_id`

These four identify the app to Firebase and are not secrets. `google-services.json` can then be deleted. 8. Restart the backend (`dev-down.ps1` , then `dev-up.ps1`). - Its window says *Push notifications are on (Firebase project …)*. - http://localhost:3000/api/v1/health/ready shows `"push":"on"` . 9. Run the app with the Firebase settings:

```
powershell cd 1-presentation-layer\mobile-app flutter run --dart-define=API_BASE_URL=http://10.0.2.2:3000 --dart-define-from-file=firebase-app.json
```

Pushes need Google Play services on the phone. The Galaxy S9+ emulator has them (a *Google APIs* image), and so does every phone with the Play Store.

To check it, follow *Push notifications* in how-to-test.md. In short: sign in on the phone as a patient whose account is linked, leave the app, then add a screening record for that patient as a clinician. *New screening record* appears on the phone within a few seconds.

To switch it off, empty `FCM_SERVICE_ACCOUNT_FILE` and restart the backend.

If the backend stops at start-up with a message that begins *FCM_SERVICE_ACCOUNT_FILE*, the path is wrong or the file is not the service-account key from step 5. The message says which; it never shows the key.